

Boosting LLM Serving through Spatial-Temporal GPU Resource Sharing

Zeja Lin
Sun Yat-sen University
Guangzhou, China
linzj39@mail2.sysu.edu.cn

Hongxin Xu
Sun Yat-sen University
Guangzhou, China
xuhx56@mail2.sysu.edu.cn

Guanyi Chen
Sun Yat-sen University
Guangzhou, China
chengy259@mail2.sysu.edu.cn

Zhiguang Chen
Sun Yat-sen University
Guangzhou, China
chenzhg29@mail.sysu.edu.cn

Yutong Lu
Sun Yat-sen University
Guangzhou, China
luyutong@mail.sysu.edu.cn

Xianwei Zhang*
Sun Yat-sen University
Guangzhou, China
zhangxw79@mail.sysu.edu.cn

Abstract

Modern large language model (LLM) serving systems confront inefficient GPU utilization due to the fundamental mismatch between compute-intensive prefill phase and memory-bound decode phase. While current practices attempt to address this by organizing these phases into hybrid batches, such solutions create an inefficient tradeoff that sacrifices either throughput or latency, leaving substantial GPU resources underutilized. For this, we identify two key root causes: 1) the prefill phase suffers from suboptimal compute utilization due to wave quantization and attention bottlenecks, and 2) hybrid batching disproportionately prioritizes latency over throughput, wasting both compute resources and memory bandwidth. To mitigate the issues, we present *Bullet*, a novel spatial-temporal orchestration system that eliminates these inefficiencies through fine-grained phase coordination. *Bullet* enables concurrent execution of prefill and decode requests, while dynamically provisioning GPU resources based on real-time performance modeling. By integrating SLO-aware scheduling and adaptive resource allocation, *Bullet* maximizes GPU utilization without compromising latency targets. Experimental evaluations on real-world workloads demonstrate that *Bullet* delivers 1.26× average throughput gains (up to 1.55×) over state-of-the-arts, while consistently meeting latency constraints.

1 Introduction

GPUs have become the predominant computing platform for large language model (LLM) services [21, 24, 41], powering a wide range of applications with varying computational and latency demands [1, 31]. As these applications continue to grow in scale and complexity, maximizing GPU utilization has become crucial for elevating service quality [37, 60]. In response, a plethora of systems have been developed to optimize different aspects of LLM serving, such as kernel-level performance [20, 35, 44], scheduling strategies [6, 43, 59], and parallelization techniques [51].

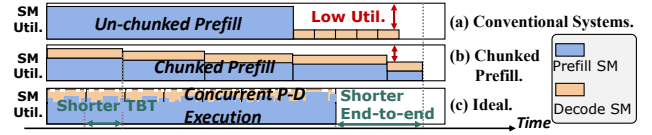


Figure 1. Request-level computation of conventional systems (top), chunked prefill (middle) and ideal (bottom).

However, the divergent computational characteristics of LLM inference make high GPU resource utilization particularly challenging [60]. In detail, the workflow consists of a computationally intensive *prefill* phase that processes all inputs in parallel, succeeded by a memory-bound *decode* phase that generates tokens sequentially. These two contrasting phases lead to a structural imbalance in the utilization of computational resources and memory bandwidth. The dynamic nature of workload patterns further exacerbates this imbalance, forcing GPUs to alternate between these complementary resource utilization states. Moreover, the distinct characteristics of the two phases inherently introduce a **throughput-latency tradeoff** [6]. For example, conventional systems [35] (Figure 1a) prioritize the prefill, which thus increases the size of subsequent decode batches. While this improves throughput, it also penalizes latency, since prefill operations monopolize GPU resources. Therefore, an ideal system necessitates fine-grained control to balance throughput and latency without elevated GPU utilization, and is non-trivial to achieve.

Prior attempts have sought to address phase imbalance either by isolating prefill and decode across different GPUs, or by coordinating them on the same device through *spatial* multiplexing. Prefill-decode disaggregation [21, 43, 59] represents the first approach, physically separating the two phases across dedicated GPU groups. However, such systems require careful tuning of GPU allocations for each phase, tailored to specific workload patterns [21], and struggle to reconfigure quickly under fluctuating request loads [59]. Furthermore, they impose heavy demands on high-bandwidth interconnects to support frequent state migration [45], significantly restricting the deployment scenarios. In practice, chunked

*Corresponding author.

通过时空GPU资源共享促进LLM服务

Zejia Lin
Sun Yat-sen University
Guangzhou, China
linzj39@mail2.sysu.edu.cn

Zhiguang Chen
Sun Yat-sen University
Guangzhou, China
chenzhg29@mail.sysu.edu.cn

Hongxin Xu
Sun Yat-sen University
Guangzhou, China
xuhx56@mail2.sysu.edu.cn

Yutong Lu
Sun Yat-sen University
Guangzhou, China
luyutong@mail.sysu.edu.cn

Guanyi Chen
Sun Yat-sen University
Guangzhou, China
chengy259@mail2.sysu.edu.cn

Xianwei Zhang*
Sun Yat-sen University
Guangzhou, China
zhangxw79@mail.sysu.edu.cn

抽象的

由于计算密集型预填充阶段和内存限制解码阶段之间的根本不匹配，现代大语言模型（LLM）服务系统面临着GPU利用率低下的问题。虽然当前的做法试图通过这些阶段组织成混合批次来解决这个问题，但此类解决方案会造成低效的权衡，牺牲吞吐量或延迟，从而导致大量GPU资源未得到充分利用。为此，我们确定了两个关键根本原因：1）由于波量化和注意力瓶颈，预填充阶段的计算利用率不佳；2）混合批处理不成比例地优先考虑延迟而不是吞吐量，浪费了计算资源和内存带宽。为了缓解这些问题，我们提出了Bullet，这是一种新颖的时空编排系统，可以通过细粒度的阶段协调消除这些低效率问题。Bullet支持并行执行预填充和解码请求，同时基于实时性能建模动态配置GPU资源。通过集成SLO感知调度和自适应资源分配，Bullet在不影响延迟目标的情况下最大限度地提高了GPU利用率。对实际工作负载的实验评估表明，与最先进的技术相比，Bullet的平均吞吐量提高了1.26×（高达1.55×），同时始终满足延迟限制。

1 简介

GPU已成为大型语言模型（LLM）服务的主要计算平台[21, 24, 41]，为具有不同计算和延迟需求的各种应用程序提供支持[1, 31]。随着这些应用程序的规模和复杂性不断增长，最大化GPU利用率对于提高服务质量变得至关重要[37, 60]。为此，人们开发了大量的系统来优化LLM服务的不同方面，例如内核级性能[20, 35, 44]、调度策略[6, 43, 59]和并行化技术[51]。

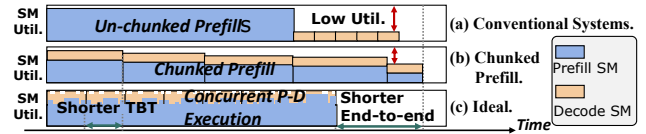


图 1. 传统系统（上）、分块预填充（中）和理想系统（下）的请求级计算。

然而，LLM推理的不同计算特征使得GPU资源利用率特别具有挑战性[60]。具体来说，该工作流程包括一个计算密集型预填充阶段，该阶段并行处理所有输入，随后是一个内存绑定解码阶段，该阶段顺序生成令牌。这两个截然不同的阶段导致计算资源和内存带宽的利用出现结构性不平衡。工作负载模式的动态性质进一步加剧了这种不平衡，迫使GPU在这些互补的资源利用状态之间交替。此外，两个阶段的独特特征本质上引入了吞吐量-延迟权衡[6]。例如，传统系统[35]（图1a）优先考虑预填充，从而增加了后续解码批次的大小。虽然这提高了吞吐量，但也会降低延迟，因为预填充操作会独占GPU资源。因此，理想的系统需要细粒度的控制来平衡吞吐量和延迟，而不提高GPU利用率，而且实现起来并不容易。

之前的尝试试图通过隔离不同GPU之间的预填充和解码，或者通过空间复用在同一设备上协调它们来解决相位不平衡问题。预填充解码分解[21, 43, 59]代表第一种方法，在物理上将两个阶段分离到专用GPU组上。然而，此类系统需要仔细调整每个阶段的GPU分配，根据特定的工作负载模式进行定制[21]，并且难以在请求负载波动的情况下快速重新配置[59]。此外，它们对高带宽互连提出了很高的要求，以支持频繁的状态迁移[45]，极大地限制了部署场景。在实践中，分块

*Corresponding author.

prefill [6] (Figure 1b) has been pervasively adopted in production systems [35, 38, 47, 58]. This method leverages a fixed token budget to combine prefill and decode requests into *hybrid batches*, with longer sequences being partitioned into chunks to fit within capacity. While tunable chunk sizes provide a degree of latency control, the approach inevitably compromises throughput [6, 44]. Smaller chunk sizes often underutilize GPU capacity, while larger ones negate latency improvements, reflecting the inherent trade-off in balancing efficiency and responsiveness.

Although throughput-latency tradeoff in LLM serving has been extensively studied [6, 59], we identify a critical limitation that, existing approaches often underutilize GPUs, resulting in suboptimal balance between latency and hardware efficiency. First, the attention exhibits relatively low compute utilization compared to linear layers (§2.2) despite optimized implementations [20]. Second, the throughput-latency tradeoff in chunked prefill exhibits sub-linear scaling with chunk sizes that prolongs execution time for successive chunks (§2.3). Third, while spatial sharing has the potential to exploit the natural complementarity between compute-intensive prefill and memory-bound decode phases through concurrent execution (Figure 1c), effective orchestration is required. Without careful coordination, contention and latency violations remain significant risks. These observations underscore the need for dynamic coordination to fully exploit GPU performance. To address GPU under-utilization and balance the trade-off between throughput and latency, we propose *Bullet*, an LLM serving system that saturates GPU resources through **spatial-temporal orchestration with fine-grained resource provisioning**. *Bullet* proactively monitors request progress and dynamically adjusts resource provision to sustain high utilization while satisfying latency requirements. The system achieves such efficient execution with four key components: 1) a performance estimator (§3.2) with an accurate analytical model of low profile and runtime overhead; 2) an SLO-aware task scheduler (§3.3) that dynamically adjusts prefill and decode requests to balance throughput and latency; 3) a computational resource manager (§3.4) that offers lightning yet precise resource configuration; 4) a concurrent execution engine (§3.5) that enables asynchronous CPU control flow and GPU execution.

Bullet achieves optimal resource provisioning via an SM-scaling roofline model that captures contention between co-executed prefill and decode kernels with varying computational resource partitions. Calibrated with minimal samples, the model is capable of continuous refinement with online data. Layer-wise prefill scheduling monitors request progress and SLO compliance, reconfiguring resources through lightweight compute-unit masking [7]. With microsecond-level operational overhead, *Bullet* maintains both precise control and high efficiency throughout execution. *Bullet* is available at <https://github.com/zejia-lin/Bullet>.

In summary, the paper makes the following contributions:

- We identify the inefficiencies that hinder GPU utilization in existing LLM serving systems while navigating throughput-latency tradeoffs, and systematically analyze opportunities for boosting utilization.
- We establish accurate modeling for spatial-temporal shared phases, and introduce fine-grained control mechanisms for latency-aware resource provisioning.
- We design and implement an LLM serving system to effectively integrate the proposed techniques into readily available frameworks.
- Experimental evaluations on real-world workloads show that *Bullet* outperforms state-of-the-arts in both latency and throughput while achieving significantly higher GPU utilization.

2 Background and Motivation

2.1 LLM Computational Workflow

Recent LLMs [24, 41, 53] are mainly built by stacking Transformer blocks [49], with each containing four core components: QKV-projection layer, self-attention computation, Output-projection layer and multi-layer perception (MLP). These components are primarily implemented through general matrix multiplications (GEMMs), with element-wise operations interspersed between layers. Among them, the self-attention specifically operates on query, key, and value matrices produced by the QKV-projection. Its computational efficiency has been significantly enhanced through optimized kernel implementations like FlashAttention [20].

The LLM inference pipeline operates through two distinct computational phases. During the initial prefill phase, the system processes all input tokens in parallel to generate the first output token, with the latency measured as time-to-first-token (TTFT). This compute-intensive stage performs full attention computations across the entire sequence while building the KV cache to store intermediate key-value states. Subsequently, the decode phase generates tokens sequentially, with each iteration consuming only the most recent output token to produce the next. The average iteration latency is termed time-per-output-token (TPOT). Unlike prefill, this memory-bound phase primarily retrieves data from the KV cache and performs relatively lightweight computations for the current token's transformations.

2.2 GPU Utilization Principles

2.2.1 Execution Model and Theoretical Performance Bound. Modern GPUs employ a hierarchical architecture with hundreds of streaming multiprocessors (SMs), each containing general-purpose cores and specialized matrix units like Tensor Cores [17]. GPU's grid-block-thread programming model [18] aligns with this architecture, where kernels are organized into grids of thread blocks (TBs) that

预填充[6] (图1b) 已广泛应用于生产系统[35,38,47,58]。此方法利用固定的令牌预算将预填充和解码请求合并到混合批次中, 并将较长的序列划分为块以适应容量。虽然可调块大小提供一定程度的延迟控制, 但该方法不可避免地会损害吞吐量 [6, 44]。较小的块大小通常无法充分利用 GPU 容量, 而较大的块大小则无法改善延迟, 这反映了平衡效率和响应能力方面的固有权衡。

尽管 LLM 服务中的吞吐量-延迟权衡已被广泛研究 [6, 59], 但我们发现了一个关键限制, 即现有方法经常未充分利用 GPU, 导致延迟和硬件效率之间的平衡不理想。首先, 尽管有优化的实现, 但与线性层 (§2.2) 相比, 注意力的计算利用率相对较低[20]。其次, 分块预填充中的吞吐量-延迟权衡表现出与块大小的次线性缩放, 这延长了连续块的执行时间 (§2.3)。第三, 虽然空间共享有可能通过并发执行来利用计算密集型预填充和内存限制解码阶段之间的自然互补性 (图 1c), 但仍需要有效的编排。如果没有仔细协调, 争用和延迟违规仍然是重大风险。这些观察结果强调了动态协调的必要性, 以充分利用 GPU 性能。为了解决 GPU 利用率不足的问题并平衡吞吐量和延迟之间的权衡, 我们提出了 Bullet, 这是一种 LLM 服务系统, 可通过时空编排和细粒度的资源配置来饱和 GPU 资源。Bullet 主动监控请求进度并动态调整资源供应, 以维持高利用率, 同时满足延迟要求。该系统通过四个关键组件实现如此高效的执行: 1) 性能估计器 (§3.2), 具有低配置和运行时开销的精确分析模型; 2) 一个SLO感知任务调度程序 (§3.3), 动态调整预填充和解码请求以平衡吞吐量和延迟; 3) 计算资源管理器 (第 3.4 节), 提供闪电且精确的资源配置; 4) 并发执行引擎 (第 3.5 节), 支持异步 CPU 控制流和 GPU 执行。

Bullet 通过 SM 扩展屋顶线模型实现最佳资源配置, 该模型捕获具有不同计算资源分区的共同执行的预填充和解码内核之间的争用。该模型使用最少的样本进行校准, 能够利用在线数据不断完善。逐层预填充调度监控请求进度和 SLO 合规性, 通过轻量级计算单元屏蔽重新配置资源 [7]。凭借微秒级的操作开销, Bullet 在整个执行过程中保持精确控制和高效率。Bullet 可在 <https://github.com/zejia-lin/Bullet> 获取。

总之, 本文做出了以下贡献:

- 我们在处理吞吐量与延迟之间进行权衡时, 发现了阻碍现有 LLM 服务系统中 GPU 使用的低效率问题, 并系统地分析了提高利用率的机会。
- 我们为时空共享阶段建立了准确的建模, 并引入了用于延迟感知资源配置的细粒度控制机制。
- 我们设计并实现了一个**大语言模型**系统, 以有效地将所提出的技术集成到现成的框架中。
- 对实际工作负载的实验评估表明, Bullet 在延迟和吞吐量方面均优于最先进的技术, 同时显著提高了 GPU 利用率。

2 背景和动机

2.1 LLM计算工作流程

最近的LLM[24,41,53]主要是通过堆叠Transformer块[49]构建的, 每个块包含四个核心组件: QKV投影层、自注意力计算、输出投影层和多层感知 (MLP)。这些组件主要通过通用矩阵乘法 (GEMM) 来实现, 并在层之间散布逐元素运算。其中, 自注意力专门对 QKV 投影生成的查询、键和值矩阵进行操作。通过 FlashAttention [20] 等优化的内核实现, 其计算效率得到了显著提高。

LLM 推理流程通过两个不同的计算阶段进行操作。在初始预填充阶段, 系统并行处理所有输入令牌以生成第一个输出令牌, 延迟时间以第一个令牌时间 (TTFT) 来衡量。这个计算密集型阶段在整个序列上执行全面的注意力计算, 同时构建 KV 缓存来存储中间键值状态。随后, 解码阶段按顺序生成令牌, 每次迭代仅消耗最近的输出令牌来生成下一个。平均迭代延迟被称为每个输出令牌的时间 (TPOT)。与预填充不同, 这个内存绑定阶段主要从 KV 缓存中检索数据, 并对当前令牌的转换执行相对轻量级的计算。

2.2 GPU利用原则

2.2.1 执行模型和理论性能界限。现代 GPU 采用具有数百个流式多处理器 (SM) 的分层架构, 每个处理器都包含通用核心和专用矩阵单元, 例如 Tensor Core [17]。GPU 的网格块线程编程模型 [18] 与该架构相一致, 其中内核被组织成线程块 (TB) 网格,

each manage thousands of cooperating threads. Upon kernel launch, it enters an asynchronous task queue (termed *stream* in CUDA[18]) for scheduling. The hardware scheduler retrieves kernels from these queues and dispatches them across SMs, enabling the concurrent kernel execution (CKE) [27, 40, 50] from different streams when required resources are satisfied.

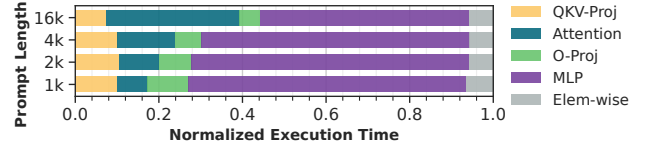
During execution, multiple TBs can reside on the same SM to share its registers, shared memory, and thread slots. The TBs per SM can be obtained via hardware vendor’s runtime APIs [2, 18]. The SM executes the *warps* (32-thread groups) in successive *wavefronts* to interleave different instructions and maximize hardware utilization. However, if the number of TBs is not evenly divisible by the number of SMs, a workload imbalance situation called *wave quantization* [15, 36] occurs. Some SMs finish early and remain idle while waiting for others to complete the tail wave. Formally, given a kernel with g TBs, N SMs, and b TBs per SM, the kernel demands $w = \lceil g/(b \cdot N) \rceil$ waves to complete. In the final wave, the TBs are distributed unevenly due to Most-Room Policy [9, 27], resulting in only $tail = \lceil g/b - N \cdot (w - 1) \rceil$ SMs active. Therefore, the corresponding ratio of idled SM cycles can be quantified as $idle = (N - tail)/(Nw)$.

GPU kernels typically use power-of-2 grid sizes to match data dimensions, but this conflicts with non-power-of-2 SM counts in GPUs. For example, 108 for Nvidia A100 [16] and 132 for H100 [17]. Therefore, wave quantization remains an open issue [34, 42] across diverse kernels. This inefficiency is particularly pronounced in Transformer’s self-attention [20, 49] and small-shaped GEMMs for short input sequences or small *chunked prefill-sizes* (§2.3.1) [6]. Additional underutilization cause arises from memory-bound kernels like LLMs’ decode phases and element-wise operators, which idle compute resources during frequent memory accesses.

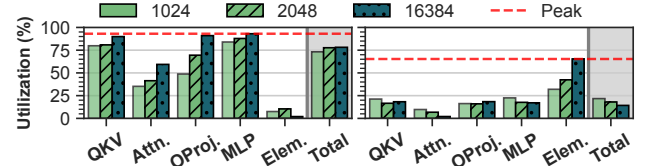
2.2.2 LLM Kernel Characteristics. We quantify LLM serving efficiency by analyzing execution time (Figure 2a), hardware utilization (Figure 2b,c), and the theoretical waste caused by wave quantization (Table 1). While MLP operations achieve up to 92% compute utilization, complete Transformer layers sustain only 70%-76% due to compounding inefficiencies. For shorter sequences, severe wave quantization in GEMMs creates substantial underutilization, as evidenced by the O-proj’s measured 49% and 70% utilization, respectively. This result closely agrees with the theoretical bound of 59% and 79% ($100 - idle\%$ in Table 1). The attention kernels,

Table 1. Theoretical SM idle ratio (%) caused by wave quantization effects, normalized to kernel/layer’s execution time.

Seq. Len.	QKV	Attn	O	MLP	Layer’s Total
1024	11.1	21.0	40.7	13.0	19.4
2048	11.1	5.2	21.0	7.6	10.4
4096	11.1	5.2	5.2	7.6	9.1
16384	1.9	0.2	0.2	0.4	0.5



(a) Execution time. (CPU overhead excluded)

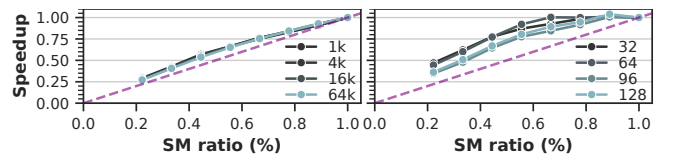


(b) Compute utilization. (c) Memory bandwidth utilization.

Figure 2. Breakdown of execution time and hardware utilization in the prefill phase of Llama-3.1-8B model on Nvidia A100 GPU. The aggregate utilization per layer remains below peak sustainable capacity (red line).

even with optimized implementations [20], exhibit lower utilization than GEMM and account for 34% of the runtime on long sequences. Together, the effects of wave quantization and attention bottleneck create persistent performance gaps between theoretical peak and achieved throughput. These gaps remain substantial regardless of sequence length, inherently constraining overall system efficiency.

2.2.3 GPU Resource Provisioning and Sharing. Naturally, compute- and memory-bound kernels are suitable to co-execute on GPUs, saturating both compute and bandwidth resources [12, 29, 46, 56]. The complementary nature of the prefill and decode phases makes them ideal for such concurrent execution. Since LLM serving systems generally necessitate adherence to service-level objectives (SLOs) of predefined latency requirements [6, 43, 59], predictable and controllable execution time over the two phases is demanded. However, current GPUs lack deterministic concurrent scheduling controls [8, 33, 40], forcing users to carefully provision resources for kernels to achieve reliable overlap [46, 56, 57]. While modern GPUs provide compute resource partitioning through Nvidia’s multi-process service (MPS) [39], precise kernel management is still required to ensure effective resource sharing while meeting SLO requirements. Figure 3 demonstrates that prefill scales near-linearly with SM count, while decode exhibits super-linear scaling. This suggests potential throughput gains from concurrent execution with properly balanced SM allocation.



(a) Prefill phase. (b) Decode phase. (CL: 2048)

Figure 3. Speedup of using partial SMs normalized to using full GPU. (Purple dashed line: linear scale.)

每个管理着数千个协作线程。内核启动后，它进入异步任务队列（在 CUDA[18] 中称为流）进行调度。硬件调度器从这些队列中检索内核并在 SM 之间分派它们，从而在满足所需资源时启用来自不同流的并发内核执行（CKE）[27,40,50]。

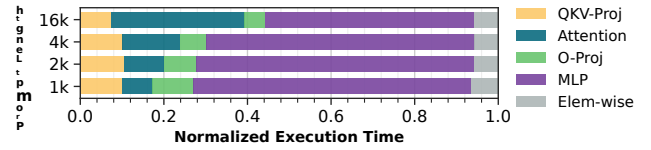
在执行期间，多个TB可以驻留在同一个SM上以共享其寄存器、共享存储器和线程槽。每个 SM 的 TB 可以通过硬件供应商的运行时 API 获得 [2, 18]。SM 在连续的波前执行扭曲（32 线程组）以交错不同的指令并最大化硬件利用率。然而，如果TB的数量不能被SM的数量整除，就会出现称为波量化[15, 36]的工作负载不平衡情况。一些 SM 提前完成并保持空闲状态，等待其他人完成尾波。形式上，给定一个具有 g TB、 N SM 和每个 SM b TB 的内核，内核需要 $w = \lceil g/(b \cdot N) \rceil$ 波来完成。在最后一波中，由于大多数房间策略 [9, 27]，TB 分布不均匀，导致只有 $tail = \lceil g/b - N \cdot (w - 1) \rceil$ SM 处于活动状态。因此，相应的空闲SM周期比例可以量化为 $idle = (N - tail)/(Nw)$ 。

GPU 内核通常使用 2 的幂网格大小来匹配数据维度，但这与 GPU 中的非 2 幂 SM 计数冲突。例如，Nvidia A 100 [16] 为 108，H100 [17] 为 132。因此，波量化在不同的内核中仍然是一个悬而未决的问题 [34, 42]。这种低效率在 Transformer 的自注意力 [20, 49] 和用于短输入序列或小分块预填充大小 (§2.3.1) [6] 的小型 GEMM 中尤其明显。其他利用率不足的原因来自于内存限制的内核，例如 LLM 的解码阶段和逐元素运算符，它们在频繁的内存访问期间闲置了计算资源。

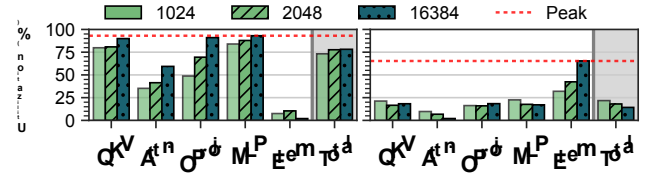
2.2.2 LLM 核心特征。我们通过分析执行时间（图 2a）、硬件利用率（图 2b、c）以及波量化造成的理论浪费（表 1）来量化 LLM 服务效率。虽然 MLP 操作实现了高达 92% 的计算利用率，但由于复合效率低下，完整的 Transformer 层仅维持 70%-76%。对于较短的序列，GEMM 中严重的波量化会造成严重的利用率不足，O-proj 测得的利用率分别为 49% 和 70%。该结果与理论界限 59% 和 79%（表 1 中的 $100-idle\%$ ）非常吻合。注意力内核，

表 1. 由波量化效应引起的理论 SM 空闲率 (%), 标准化为内核/层的执行时间。

Seq. Len.	QKV	Attn	O	MLP	Layer's Total
1024	11.1	21.0	40.7	13.0	19.4
2048	11.1	5.2	21.0	7.6	10.4
4096	11.1	5.2	5.2	7.6	9.1
16384	1.9	0.2	0.2	0.4	0.5



(a) Execution time. (CPU overhead excluded)

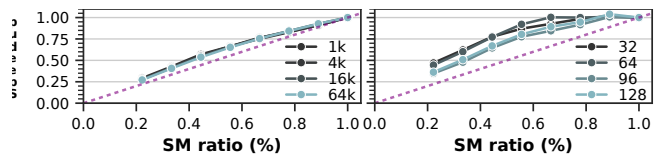


(b) Compute utilization. (c) Memory bandwidth utilization.

图 2. Nvidia A100 GPU 上 Llama-3.1-8B 模型预填充阶段的执行时间和硬件利用率的细分。每层的总利用率仍低于峰值可持续容量（红线）。

即使采用优化的实现 [20]，其利用率也低于 GEMM，并且在长序列上占运行时间的 34%。波量化和注意力瓶颈的影响共同造成理论峰值和实现的吞吐量之间持续的性能差距。无论序列长度如何，这些差距仍然很大，本质上限制了整个系统的效率。

2.2.3 GPU 资源配置和共享。当然，计算和内存限制的内核适合在 GPU 上共同执行，从而使计算和带宽资源饱和 [12,29,46,56]。预填充和解码阶段的互补性质使其成为此类并发执行的理想选择。由于 LLM 服务系统通常需要遵守预定义延迟要求的服务级别目标 (SLO) [6,43,59]，因此需要两个阶段的可预测且可控的执行时间。然而，当前的 GPU 缺乏确定性并发调度控制 [8,33,40]，迫使用户仔细为内核提供资源以实现可靠的重叠 [46,56,57]。虽然现代 GPU 通过 Nvidia 的多进程服务 (MPS) [39] 提供计算资源分区，但仍然需要精确的内核管理，以确保有效的资源共享，同时满足 SLO 要求。图 3 表明预填充随 SM 计数近乎线性缩放，而解码呈现超线性缩放。这表明通过适当平衡的 SM 分配并发执行可以带来潜在的吞吐量增益。



(a) Prefill phase. (b) Decode phase. (CL: 2048)

图 3. 使用部分 SM 归一化为使用完整 GPU 的加速。（紫色虚线：线性刻度。）

Takeaway 1. GPUs remain underutilized even during compute-intensive prefill. While co-locating prefill and decode saturates the resources, precise resource provisioning to orchestrate the two phases is demanded.

2.3 Biased Throughput-Latency Tradeoff

2.3.1 Chunked Prefill Workflow. As shown in Figure 4, chunked prefill [6] achieves low TPOT by leveraging a fixed token budget to concatenate the prefill and decode tokens into a *hybrid batch*, executing in a lockstep fashion (②). Given a chunk size of cs , the hybrid batch is first filled with ds active decode requests first, and allocates the remaining $cs - ds$ tokens to the prefill sequences. Sequences sl exceeding this residual capacity are split into chunks, leaving residual tokens processed in subsequent iterations. Therefore, the prefill completion requires $N = \lceil sl / (cs - ds) \rceil$ iterations. This forces $N \cdot (N + 1) / 2$ times KV cache reloads as each new chunk *must* reprocess previous chunks' cached states.

Due to the lock-step execution of the hybrid batch, a smaller chunk size effectively decreases TPOT at the cost of increased TTFT and degraded system throughput [6], while larger chunks exhibit the opposite effect. Previous works [6, 60] recognize these inherent tradeoffs and propose tuning chunk size based on workload's prefill-to-decode time ratio through manual tuning or automatic searching [5, 11].

2.3.2 Sub-optimal Hardware Utilization. Despite the throughput-latency tradeoff has been extensively studied, we highlight that such a tradeoff is biased, and the resulting performance degradation remains overlooked. **First**, as discussed in §2.2.2, chunked prefill typically uses suboptimal chunk sizes below GPU-saturating levels to prioritize low latency. This produces severe wave quantization effects [42], creating GPU bubbles (Figure 4-①). **Second**, redundant KV cache reloads required for long sequences significantly prolong attention computation time (③), further reducing GPU utilization. **Third**, these factors collectively inflate TTFT, triggering a cascading congestion effect in which queued requests stall while awaiting prefill completion, degrading overall system throughput.

Figure 5 systematically quantify the performance degradation of chunked prefill of a 16k-token sequence prefill even *without* hybrid batching decode requests. For 1k chunk

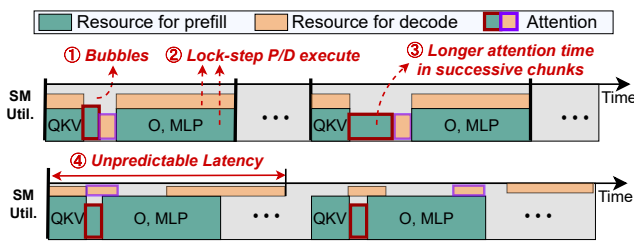
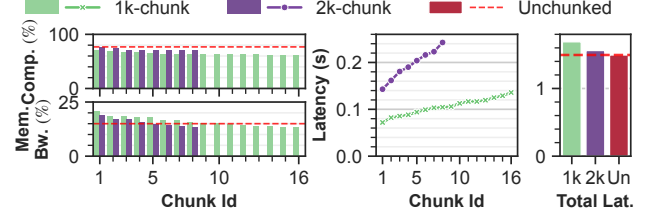


Figure 4. Kernel-level workflow of existing systems featuring chunked (top) and uncoordinated sharing (bottom). Both suffer from biased throughput-latency tradeoff.



(a) Hardware utilization.

(b) Per-chunk and total latency.

Figure 5. Per-chunk GPU utilization and latency across varying chunk sizes, all exhibiting degraded throughput.

size, a progressive 10% drop in compute efficiency (from 71% to 61%) across successive chunks is witnessed in Figure 5a, which falls substantially below the 77% achievable peak. This under-utilization stems from redundant KV cache reloading in chunked attention, which also causes the final chunk's processing time to 1.9× than that of the initial chunk. Consequently, per-chunk latency scales linearly by chunk counts and increases total prefill latency by 1.13× compared to unchunked execution. While a larger chunk size of 2k partially mitigates utilization drops from -18% to -7%, the average per-chunk latency increases by 1.86×, significantly diminishing the TPOT improvements that motivated chunked prefill. This fundamental tension between maintaining high hardware utilization and minimizing TPOT creates an intractable optimization in chunked prefill.

2.3.3 Existing Optimizations. Despite recent optimizations of chunked prefill, the technique remains intrinsically limited. PodAttention [44] fuses prefill and decode attention kernels to accelerate hybrid-batch attention, yet this operator-specific optimization does not mitigate the fundamental latency penalties imposed by chunking (§2.3.2). Nanoflow [60] designs a fixed-size pipeline that uses CUDA streams to overlap chunked attention and linear layers with careful inter-kernel synchronizations. However, the latency of chunked attention grows by successive chunks, which eventually eliminates the chances of overlapping. Apart from chunked prefill-based approaches, MuxServe [23] (Figure 4) decouples prefill and decode phases into separate processes, leveraging MPS [39] with manually configured, fixed SM quotas to enable spatial sharing. This coarse-grained static

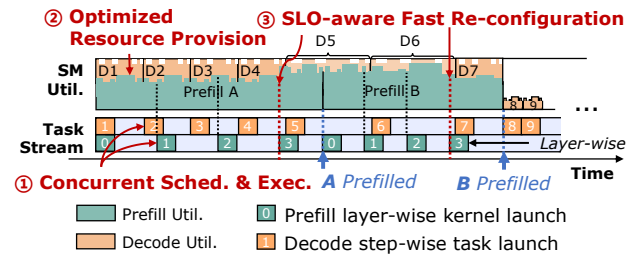


Figure 6. Dynamic spatial-temporal orchestration of concurrently executed prefill and decode tasks.

要点 1. 即使在计算密集型预填充期间, GPU 仍未得到充分利用。虽然预填充和解码共置会使资源饱和, 但需要精确的资源配置来协调这两个阶段。

2.3 有偏差的吞吐量-延迟权衡

2.3.1 分块预填充工作流程。如图 4 所示, 分块预填充 [6] 通过利用固定的令牌预算将预填充和解码令牌连接到混合批次中, 以锁步方式执行 (2), 从而实现低 TPOT。给定块大小为 cs , 混合批次首先填充 ds 活动解码请求, 并将剩余的 $cs - ds$ 令牌分配给预填充序列。超过此剩余容量的序列 sl 被分成块, 留下在后续迭代中处理的剩余标记。因此, 预填充完成需要 $N = \lceil sl / (cs - ds) \rceil$ 次迭代。这会强制 $N \cdot (N + 1) / 2$ 次 KV 缓存重新加载, 因为每个新块必须重新处理先前块的缓存状态。

由于混合批处理的锁步执行, 较小的块大小有效地降低了 TPOT, 但代价是增加了 TTFT 并降低了系统吞吐量 [6], 而较大的块则表现出相反的效果。之前的工作 [6, 60] 认识到这些固有的权衡, 并建议通过手动调整或自动搜索来根据工作负载的预填充与解码时间比率来调整块大小 [5, 11]。

2.3.2 次优硬件利用率。尽管吞吐量-延迟权衡已被广泛研究, 但我们强调这种权衡是有偏差的, 并且由此产生的性能下降仍然被忽视。首先, 如第 2.2.2 节中所述, 分块预填充通常使用低于 GPU 饱和水平的次优块大小来优先考虑低延迟。这会产生严重的波量化效应 [42], 从而产生 GPU 气泡 (图 4-1)。其次, 长序列所需的冗余 KV 缓存重新加载显著延长了注意力计算时间 (3), 进一步降低了 GPU 利用率。第三, 这些因素共同导致 TTFT 膨胀, 引发级联阻塞效应, 排队请求在等待预填充完成时停滞, 从而降低整体系统吞吐量。

图 5 系统地量化了 16k 令牌序列预填充的分块预填充的性能下降, 即使没有混合批处理解码请求。对于 1k 块

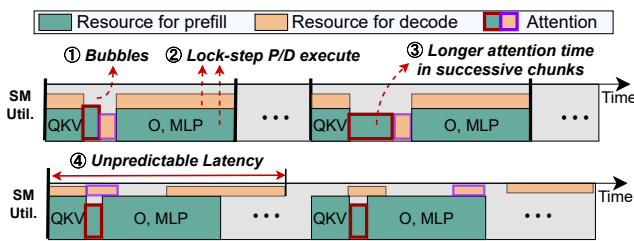


图 4. 现有系统的内核级工作流程, 具有分块 (顶部) 和不协调共享 (底部) 的特点。两者都遭受吞吐量与延迟权衡的偏差。

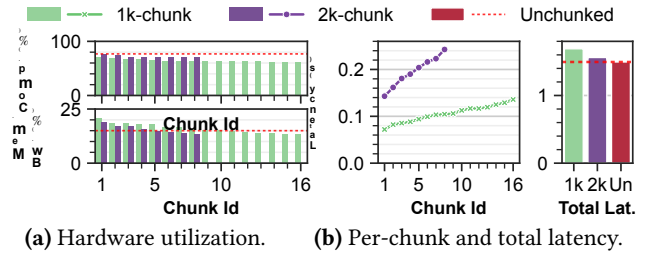


图 5. 不同块大小的每个块 GPU 利用率和延迟, 所有这些都表现出吞吐量下降。

随着大小的增加, 图 5a 中可以看到连续块的计算效率逐渐下降 10% (从 71% 到 61%), 大大低于可实现的 77% 峰值。这种利用率不足源于分块注意力中冗余的 KV 缓存重新加载, 这也导致最终块的处理时间比初始块低 1.9x。因此, 与未分块执行相比, 每块延迟会根据块计数线性扩展, 并使总预填充延迟增加 1.13x。虽然 2k 的较大块大小部分缓解了利用率从 -18% 下降到 -7%, 但平均每个块延迟增加了 1.86x, 显着削弱了推动分块预填充的 TPOT 改进。保持高硬件利用率和最小化 TPOT 之间的这种根本矛盾在分块预填充中造成了棘手的优化。

2.3.3 现有优化。尽管最近对分块预填充进行了优化, 但该技术本质上仍然有限。PodAttention [44] 融合了预填充和解码注意力内核来加速混合批量注意力, 但这种特定于操作符的优化并不能减轻分块带来的基本延迟损失 (第 2.3.2 节)。Nanoflow [60] 设计了一个固定大小的管道, 它使用 CUDA 流来重叠分块注意力层和线性层, 并进行仔细的内核间同步。然而, 分块注意力的延迟会随着连续的分块而增长, 最终消除了重叠的机会。除了基于分块预填充的方法之外, MuxServe [23] (图 4) 将预填充和解码阶段解耦为单独的进程, 利用 MPS [39] 和手动配置的固定 SM 配额来实现空间共享。这种粗粒度静态

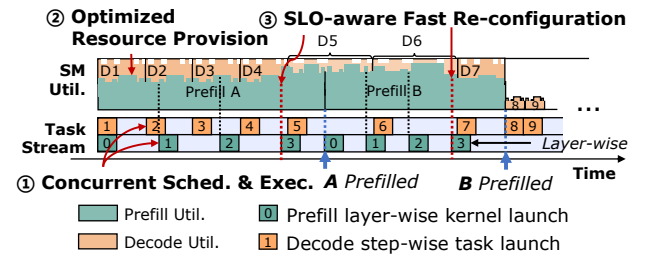


图 6. 并发执行的预填充和解码任务的动态时空编排。

partitioning leads to unpredictable latency despite saturating the GPU. Therefore, neither solution fully resolves the throughput-latency tradeoff under varying serving demands.

Takeaway-2. *Both chunked prefill and uncoordinated spatial sharing exhibit skewed throughput-latency tradeoffs. This urges for a fine-grained yet predictable execution mechanism for optimal LLM serving performance.*

2.4 Opportunities and Challenges

To address the inefficiencies discussed above, an inter-phase orchestration system is demanded to achieve optimal throughput-latency balance while maximizing GPU utilization, as illustrated in Figure 6.

Concurrent Schedule and Execution. Co-executing prefill and decode eliminates redundant KV-cache reloads and fully utilizes GPU resources, but sustaining latency targets demands real-time request monitoring and kernel scheduling that respond to execution progress and system state. A layer-level, SLO-aware scheduler must tightly control execution advance to avoid inter-phase resource competition. While LLM online serving further requires low-overhead control plane and support for rapid reconfiguration, rendering these challenges non-trivial.

Optimized Resource Provision. By carefully partitioning available SM resources, memory-bound decode kernels may efficiently co-execute with compute-intensive prefill kernels. However, LLM’s multi-dimensional nature of input parameters and dynamic inter-kernel contention creates a complex optimization space. This necessitates precise latency modeling across varying SM provisions and efficient exploration algorithms to identify optimal configurations.

SLO-aware Rapid Re-configuration. In response to runtime workload dynamics, resource allocation between prefill and decode phases must be re-configured. These frequent adjustments demand near-zero-overhead reconfiguration to maintain efficiency. Although MPS [39] offers context APIs [18] to adjust different SM quotas, it incurs significant memory overhead when applied to LLM serving scenarios due to dynamic inputs. An even more lightweight approach is demanded to maintain for instantaneous switching between optimized resource partitions.

3 Design

3.1 Overview

Figure 7 illustrates Bullet’s workflow for concurrent prefill-decode execution with dynamic and fine-grained resource provisioning, with all components operating at microsecond-level overhead. Bullet comprises four key components around the scheduling, resource partitioning and execution: performance estimator, SLO-aware task scheduler, resource manager, and concurrent execution engine. The *performance estimator* (§3.2) ❶ first builds an analytical model for the served LLM, augmented with lightweight offline samples. The model

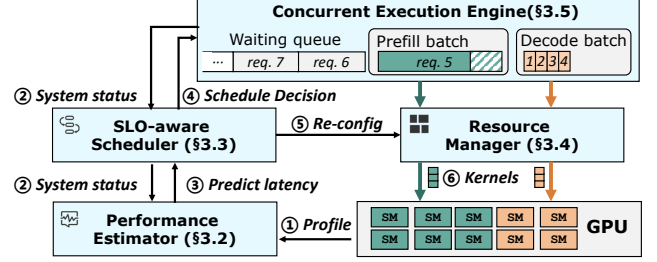


Figure 7. Workflow of Bullet. (Numbers: dataflow order.)

provides precise latency predictions across different configurations of co-executing prefill and decode batch sizes under varying resource allocations. At runtime, the *SLO-aware task scheduler* (§3.3) acts as the central coordinator to enable GPU sharing between prefill and decode tasks via spatial-temporal scheduling. During every layer-wise scheduling cycle, the scheduler ❷ proactively retrieves system status from the *concurrent execution engine* (§3.5), monitoring request progress and ❸ evaluating potential SLO violations by *performance estimator*. Observed statistics are also used to refine the performance estimator online. The scheduler rapidly searches for ❹ an optimal resource configuration and scheduling decision that maximizes throughput while ensuring SLO compliance. *Computational resource manager* is ❺ triggered for lightning resource reconfiguration when necessary. Finally, the prefill and decode kernels are ❻ launched concurrently on the provisioned SMs. Bullet dynamically balancing the competing demands of TTFT and TPOT maintains high utilization with negligible overhead.

3.2 Performance Estimator

3.2.1 Problem Formulation. For a given LLM and hardware, the latency of co-executed prefill and decode is determined by six factors, termed as Execution State (*ES*): prefill sequence length (sl_i), batch size (pbs) and number of allocated SMs (pm), alongside decode context length (cl_i), batch size (dbs) and SMs (dm). Enumerating the millions of *ES* combinations for profiling is infeasible. Therefore, we build a lightweight analytical model with a minimal profile and runtime overhead. First, we propose SM-scaling roofline model (SRM) to derive single kernel latency under partitioned SMs without interference. Second, memory-subsystem contention is quantified when concurrent kernels are isolated in distinct SMs. Finally, the model is augmented with minimal sampled data for calibration. At runtime, the model executes within microseconds and continuously refines with online statistics.

3.2.2 SM-scaling Roofline Model (SRM). We examine the performance of compute, memory access, and network communication when only N_p of the SMs are available. Theoretical compute performance scales linearly as $C_p = C_{peak} \cdot N_p / N$. Memory and network bandwidth exhibit proportional

尽管 GPU 已饱和，但分区仍会导致不可预测的延迟。因此，这两种解决方案都没有完全解决不同服务需求下的吞吐量与延迟的权衡。

外卖-2。分块预填充和不协调的空间共享都表现出倾斜的吞吐量-延迟权衡。这迫切需要一种细粒度且可预测的执行机制，以实现最佳的 LLM 服务性能。

2.4 机遇与挑战

为了解决上面讨论的低效率问题，需要一个阶段间编排系统来实现最佳的吞吐量-延迟平衡，同时最大化 GPU 利用率，如图 6 所示。

并发计划和执行。共同执行预填充和解码消除了冗余的 KV 缓存重新加载并充分利用 GPU 资源，但维持延迟目标需要实时请求监控和内核调度来响应执行进度和系统状态。层级、SLO 感知的调度程序必须严格控制执行进度，以避免阶段间资源竞争。虽然 LLM 在线服务进一步需要低开销的控制平面并支持快速重新配置，但这些挑战并非微不足道。

优化资源供给。通过仔细划分可用的 SM 资源，内存限制的解码内核可以与计算密集型预填充内核有效地共同执行。然而，LLM 输入参数的多维性质和动态内核间争用创建了一个复杂的优化空间。这需要跨不同 SM 规定进行精确的延迟建模，并需要有效的探索算法来识别最佳配置。

SLO 感知快速重新配置。为了响应运行时工作负载动态，必须重新配置预填充和解码阶段之间的资源分配。这些频繁的调整需要近乎零开销的重新配置以保持效率。尽管 MPS [39] 提供了上下文 API [18] 来调整不同的 SM 配额，但由于动态输入，当应用于 LLM 服务场景时，它会产生大量的内存开销。需要一种更轻量级的方法来维持优化资源分区之间的即时切换。

3 设计

3.1 概述

图 7 说明了 Bullet 的并发预填充解码执行工作流程，具有动态和细粒度的资源配置，所有组件都以微秒级的开销运行。Bullet 包含围绕调度、资源分区和执行的四个关键组件：性能估计器、SLO 感知任务调度器、资源管理器和并发执行引擎。性能估计器 (§3.2) ① 首先为所服务的 LLM 构建一个分析模型，并使用轻量级离线样本进行增强。型号

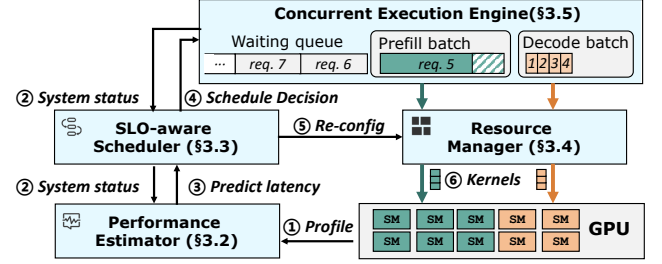


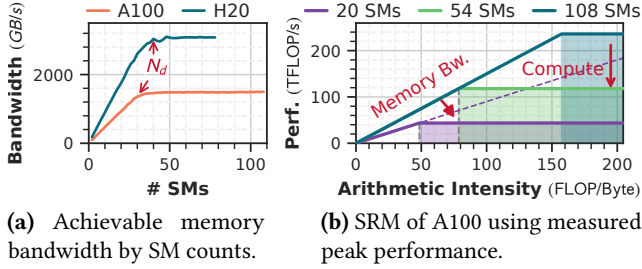
图 7. Bullet 的工作流程。（数字：数据流顺序。）

在不同的资源分配下，跨共同执行预填充和解码批量大小的不同配置提供精确的延迟预测。在运行时，SLO 感知任务调度程序 (§3.3) 充当中央协调器，通过时空调度实现预填充和解码任务之间的 GPU 共享。在每个分层调度周期中，调度程序 ② 主动从并发执行引擎（第 3.5 节）检索系统状态，监视请求进度并 ③ 通过性能估计器评估潜在的 SLO 违规。观察到的统计数据还用于在线改进性能估计器。调度程序快速搜索 ④ 最佳资源配置和调度决策，以最大限度地提高吞吐量，同时确保 SLO 合规性。⑤ 必要时触发计算资源管理器以重新配置闪电资源。最后，预填充和解码内核在配置的 SM 上同时启动。Bullet 动态平衡 TTFT 和 TPOT 的竞争需求，以可忽略不计的开销保持高利用率。

3.2 性能估算器

3.2.1 问题表述。对于给定的 LLM 和硬件，共同执行的预填充和解码的延迟由六个因素决定，称为执行状态 (ES)：预填充序列长度 (sl_i)，批量大小 (pbs) 和分配的 SM 数量 (pm)，以及解码上下文长度 (cl_i)，批量大小 (db_s) 和 SM (dm)。枚举数百万个 ES 组合进行分析是不可行的。因此，我们构建了一个具有最小配置文件和运行时开销的轻量级分析模型。首先，我们提出 SM 扩展屋顶线模型 (SRM) 来推导分区 SM 下无干扰的单内核延迟。其次，当并发内核被隔离在不同的 SM 中时，内存子系统争用就会被量化。最后，用最少的采样数据增强模型以进行校准。在运行时，模型在几微秒内执行，并通过在线统计数据不断完善。

3.2.2 SM 缩放屋顶线模型 (SRM)。当只有 N_p 个 SM 可用时，我们检查计算、内存访问和网络通信的性能。理论计算性能按 $C_p = C_{peak} \cdot N_p / N$ 线性扩展。内存和网络带宽成正比



(a) Achievable memory bandwidth by SM counts. (b) SRM of A100 using measured peak performance.

Figure 8. Modeling peak performance by number of SMs.

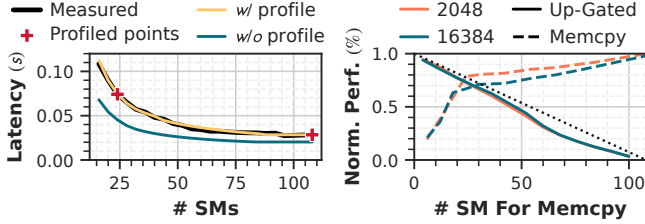


Figure 9. Calibrating estimated decode latency with only 2 samples.

Figure 10. Normalized performance of co-executed UG and memory copy.

scaling until reaching inflection points N_d and N_w , where the SMs generate sufficient traffic to saturate the respective peak bandwidth D_{peak} and W_{peak} . Figure 8a illustrates the throughput of a memory-copy kernel, showing the inflection points of Nvidia A100 and H20 GPUs.

Given a kernel with $flop_k$ operations and mem_k bytes of memory transactions, we construct an SM-scaling roofline model (SRM) in Equation 1 to estimate the theoretical latency on N_p SMs. In the example of Figure 8b, the memory bandwidth inflection point $N_d = 30$. When using 54 SMs, the attainable memory bandwidth remains at its peak, maintaining the original roofline slope while lowering the plateau. For 20 SMs, both the slope and plateau decline.

$$\begin{cases} T'_{k,p} = flop_k \cdot \min(flop_k/mem_k \cdot D_p, C_p)^{-1} \\ C_p = C_{peak} \cdot N_p/N; D_p = D_{peak} \cdot \min(1, N_p/N_d) \end{cases} \quad (1)$$

For each Execution State ES , we compute the arithmetic intensity of every LLM kernel with llm-viewer [54], apply SRM to obtain its baseline latency $T'_{k,p,ES}$ on N_p SMs, and aggregate these values to yield the total LLM latency $T'_{p,ES}$. Since practical execution rarely matches the roofline bound, Equation 2 derives a scaling factor for calibrating SRM:

$$\alpha_{p,ES} = T_{p,ES}^{measured} / T'_{p,ES} \quad (2)$$

The factor is then extrapolated to unmeasured configurations, since the utilization pattern is near-linear between similar kernel inputs [10, 12, 26]. As shown in Figure 9, only two samples are sufficient to model the decode latencies by varying SMs. This demonstrates effective estimation aligned with kernel characteristics without extensive profiling.

3.2.3 Modeling Contention. When kernels execute on isolated SMs to prevent compute contention, memory subsystem and network contention persist. We verify that kernel latency remains stable under such interference, even if underlying hardware scheduling and inter-kernel resource competition vary. Extensive evaluation of 7360 concurrent ES on A100 (serve Llama3.1-8B [24]) and 8×H20 (serve Qwen3-32B [53]), each repeated for 30 runs, observes that 95% of latency measurements deviate less than $\pm 6.8\%$ from the mean of the respective repeated runs. This tight distribution confirms the stability of kernel latency under varying prefill-decode interference patterns, which can be used as a reliable metric for contention modeling.

The worst-case memory subsystem interference can be quantified by co-executing a memory copy kernel on N_p SMs and an up-gated layer (UG), which is a large-shaped GEMM in LLM, on the residual $N - N_p$ SMs. This is because prefill and decode kernels typically exhibit lower resource utilization than both UG and memory copy. Figure 10 shows that UG exhibits marginal performance degradation ($< 8\%$) when using more than 60% SMs. Therefore, the latency of prefill kernels can be reliably calibrated via Equation 2 with minimal sample size. Conversely, memory-copy throughput decreases as UG sequence lengths grow. To model decode latencies, we measure the attainable bandwidth $D_{p,sl}$ when concurrently executed with sl -length prefill to update SRM. Equation 2 is then applied for further refinement, since the memory interference pattern scales linearly with kernel utilization characteristics [12, 52].

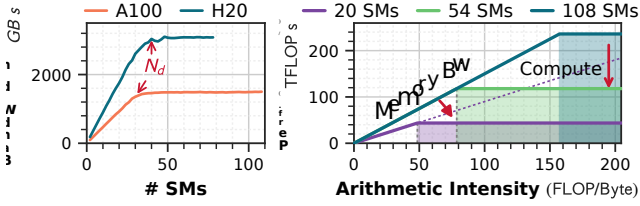
Inter-phase network contention is low since the traffic scales with either sl or dbs , where dbs is relatively small. The memory subsystem impact of network transfers is inherently lower than memory copy, and prefill-to-decode interference is already accounted for in the end-to-end $D_{p,sl}$ calibration.

3.2.4 Profiling and Online Calibration. During offline profiling, Bullet records compute, memory, and network performance across varying SM counts in a single sweep to construct SRM, then measures prefill and decode latencies for a sparse set of ES values as calibration. Continuous online data collection for recalibration is straightforward. Leveraging only Equation 2 and linear extrapolation, the runtime overhead for update and prediction is negligible.

Table 2 shows that calibrated SRM maintains high accuracy even for inputs beyond the sampled range while keeping profiling overhead low. This precision is sufficient for reliable real-time scheduling optimization.

3.3 SLO-aware Task Scheduler

3.3.1 Scheduling Workflow. Each of the prefill and decode *concurrent execution engine* (§3.5) runs a scheduler autonomously. At every step, the scheduler reads system status from the global metadata buffer (§3.5.1), forecasts latencies with *performance estimator*, and reorders pending requests.



(a) SM 计数可实现的内存带宽。 (b) 使用测得的峰值性能对 A100 进行 SRM。

图 8. 按 SM 数量对峰值性能进行建模。

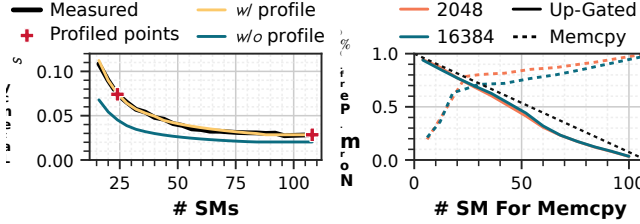


图 9. 仅使用 2 个样本校准估计的解码延迟。 图 10. 共同执行的 UG 和内存复制的标准化性能。

缩放直至达到拐点 N_d 和 N_w ，其中 SM 生成足够的流量以使各自的峰值带宽 D_{peak} 和 W_{peak} 饱和。图 8a 说明了内存复制内核的吞吐量，显示了 Nvidia A100 和 H20 GPU 的拐点。

给定具有 $flop_k$ 操作和 mem_k 字节内存事务的内核，我们在公式 1 中构建了 SM 缩放屋顶线模型（SRM），以估计 N_p SM 上的理论延迟。在图 8b 的示例中，内存带宽拐点 $N_d = 30$ 。当使用 54 个 SM 时，可达到的内存带宽保持在峰值，保持原始屋顶线斜率，同时降低平台。对于 20 个 SM，斜率和平台值均下降。

$$\begin{cases} T'_{k,p} = flop_k \cdot \min(flop_k/mem_k \cdot D_p, C_p)^{-1} \\ C_p = C_{peak} \cdot N_p/N; D_p = D_{peak} \cdot \min(1, N_p/N_d) \end{cases} \quad (1)$$

对于每个执行状态 ES ，我们使用 llm-viewer [54] 计算每个 LLM 内核的算术强度，应用 SRM 来获取其在 N_p SM 上的基线延迟 $T'_{k,p,ES}$ ，并将这些值聚合以产生总 LLM 延迟 $T'_{p,ES}$ 。由于实际执行很少与车顶线界限相匹配，因此公式 2 得出用于校准 SRM 的比例因子：

$$\alpha_{p,ES} = T_{p,ES}^{measured} / T'_{p,ES} \quad (2)$$

然后将该因子外推到未测量的配置，因为相似内核输入之间的利用率模式接近线性[10,12,26]。如图 9 所示，只需两个样本就足以通过不同的 SM 来模拟解码延迟。这证明了与内核特征一致的有效估计，而无需进行大量分析。

3.2.3 建模竞争。当内核在隔离的 SM 上执行以防止计算争用时，内存子系统和网络争用仍然存在。我们验证了内核延迟在这种干扰下保持稳定，即使底层硬件调度和内核间资源竞争发生变化。对 A100（服务 Llama3.1-8B [24]）和 8xH20（服务 Qwen3-32B [53]）上的 7360 个并发 ES 进行广泛评估，每个重复运行 30 次，观察到 95% 的延迟测量与各自重复运行的平均值的偏差小于 $\pm 6.8\%$ 。这种紧密的分布证实了在不同的预填充解码干扰模式下内核延迟的稳定性，这可以用作争用建模的可靠指标。

最坏情况下的内存子系统干扰可以通过在 N_p SM 上共同执行内存复制内核和在剩余 $N - N_p$ SM 上执行上门层（UG）（LLM 中的大型 GEMM）来量化。这是因为预填充和解码内核通常表现出比 UG 和内存复制更低的资源利用率。图 10 显示，当使用超过 60% SM 时，UG 表现出边际性能下降（ $< 8\%$ ）。因此，可以通过公式 2 以最小的样本量可靠地校准预填充内核的延迟。相反，内存复制吞吐量随着 UG 序列长度的增长而降低。为了对解码延迟进行建模，我们测量了与 sl 长度的预填充同时执行以更新 SRM 时可达到的带宽 $D_{p,sl}$ ，然后应用方程 2 进行进一步细化，因为内存干扰模式与内核利用率特性呈线性关系 [12, 52]。

阶段间网络争用较低，因为流量随 sl 或 dbs 缩放，其中 dbs 相对较小。网络传输对内存子系统的影响本质上低于内存复制，并且在端到端 $D_{p,sl}$ 校准中已经考虑了预填充到解码的干扰。

3.2.4 分析和在线校准。在离线分析过程中，Bullet 在一次扫描中记录不同 SM 数量的计算、内存和网络性能以构建 SRM，然后测量一组稀疏的 ES 值的预填充和解码延迟作为校准。用于重新校准的连续在线数据收集非常简单。仅利用公式 2 和线性外推法，更新和预测的运行开销可以忽略不计。

表 2 显示，即使输入超出采样范围，校准的 SRM 也能保持高精度，同时保持较低的分析开销。这个精度足以实现可靠的实时调度优化。

3.3 SLO感知任务调度器

3.3.1 调度工作流程。每个预填充和解码并发执行引擎（§3.5）都自主运行一个调度程序。在每一步中，调度程序都会从全局元数据缓冲区（第 3.5.1 节）中读取系统状态，使用性能估计器预测延迟，并对挂起的请求重新排序。

Table 2. Mean absolute relative error of SRM on 7360 data.

Phase (Data Range)	#Samples (Range)	MAPE
Prefill	18 ($sl \leq 16k$)	9.1%
($sl \leq 32k$)	30 ($sl \leq 32k$)	5.3%
Decode	308 ($cl \leq 16k, bs \leq 128$)	10.5%
($cl \leq 32k, bs \leq 256$)	364 ($cl \leq 32k, bs \leq 256$)	7.7%

Upon detecting potential SLO violations, the scheduler greedily searches for the optimal configuration and invokes *computational resource manager* (§3.4) to repartition SMs.

As illustrated in Figure 11, for prefill scheduler, a fixed number of layers is launched per step, and synchronizes for CPU (teal triangle) to make subsequent scheduling decisions. This enables fine-grained control over prefill progress temporarily for rapid adaptation to system fluctuations. Conversely, decode scheduler issue kernels as a single CUDA Graph [18] to eliminate the launch overhead of small kernels.

The primary scheduling objective is to prioritize prefill while respecting decode SLOs, since shorter prefill latency enlarges decode batch size and raises system throughput [6]. During concurrent operation, the decode phase is provisioned with the minimum SM counts that satisfy SLO. As the final prefill layer approaches completion, additional SMs are allocated to decode phase to facilitate a smooth transition between co-running prefill/decode and decode-only. During reconfiguration, Bullet eliminates inter-phase synchronization by partially sharing SMs between phases rather than idling unused resources. Although a perfect non-overlapping allocation is challenging, layer-wise prefill scheduling confines such compute resource interference to minimal, predictable regions.

3.3.2 Request Scheduling and Resource Provisioning.

To facilitate SLO-aware scheduling, Bullet monitors the system state defined as $S = (ES, PS, RS)$, where ES denotes the execution state (§3.2.1), PS the prefill progress, and RS the per-request latency metrics. Prefill state PS comprises the queuing-request set Q , the in-flight request set P , and the executed layer count L_{exe} . For every request i , arrival time a_i and decode-start time d_i are recorded. At time t , the prefill scheduler estimates GPU execution times for all requests in $P \cup Q$ under current ES , while the decode scheduler predicts next-step latency and updates the corresponding TPOT. These estimations are then written to the global metadata buffer, allowing each scheduler perceive the holistic state.

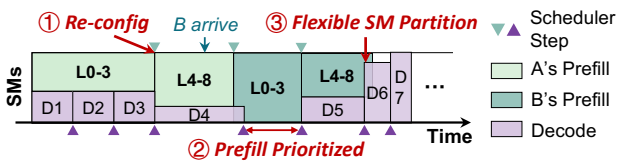


Figure 11. Asynchronous scheduler launches layer-wise prefill kernels and step-wise decode graphs, monitoring latencies and dynamically reallocating SMs.

Algorithm 1 outlines the prefill scheduler, and the decode scheduler follows analogously. The algorithm continuously monitors execution progress and updates latency estimates (lines 2-4). Requests in the waiting queue are reordered by ascending predicted latency (line 5) when the reordering does not violate TTFT SLOs for pending requests. This reduces average TTFT without starvation. To start a new prefill step (line 10-13), requests are batched until reaching arithmetic intensity limits under the current execution state. When provisioning resources, prefill phase is prioritized and provisioned with more SMs unless TPOT is compromised (line 15). In extreme cases of high request load, the decode phase would be temporarily suspended (Figure 11-②) if TPOT SLO is still met. When both SLOs cannot simultaneously be satisfied, indicating the system is beyond maximum capacity, a balanced SM ratio is enforced to limit excessive latency in either phase (line 14).

3.4 Computational Resource Manager

While MPS [39] with CUDA Green Context [3] supports SM partitioning, its memory overhead exceeds 700MB for only 4 static policies in LLM serving [19], rendering it impractical for Bullet’s fine-grained control. For flexible low-overhead SM provisioning, we employ a library of SM masking techniques [7, 8]. The `libsmctrl_set_stream_mask` API sets the metadata of a CUDA stream [18] to restrict all kernels

Algorithm 1: SLO-aware Scheduling for Prefill

Input: System state: S , TTFT/TPOT SLO: Γ_p/Γ_d

Output: Next requests & layers to run: $next_tasks$

```

1 Function Schedule( $S$ ):
2    $ttft \leftarrow \text{ESTIMATELATENCY}(S)$ 
3    $\text{WRITEGLOBALBUFFER}(ttft)$ 
4    $tpot \leftarrow \text{READGLOBALBUFFER}()$ 
5    $\text{SORTBYLEASTESTIMLATENCY}(Q)$ 
6   if  $P \neq \emptyset$  then
7      $satisfy \leftarrow \text{req}.ttft \leq \Gamma_p, \text{req} \in P$ 
8      $next\_tasks \leftarrow P$ 
9   else
10     $satisfy \leftarrow P90(ttft) \leq \Gamma_p$ 
11     $next\_tasks \leftarrow \emptyset$ 
12    while  $\text{ARITHINTEN}(next\_tasks, S.ES) < peak$  do
13       $next\_tasks.append(Q.pop())$ 
14    if not  $satisfy$  and  $P90(tpot) > \Gamma_d$  then
15       $\text{SETBALANCEDSM}(S, ttft, tpot)$ 
16    else if  $P90(tpot) \leq \Gamma_d$  then
17       $\text{REDUCEDECODESM}(S, ttft, tpot)$ 
18    else if  $P90(tpot) > \Gamma_d$  then
19       $\text{REDUCEPREFILLSM}(S, ttft, tpot)$ 
20    return  $next\_tasks, L_{exe} + L_{step}$ 

```

表 2. 7360 数据上的 SRM 平均绝对相对误差。

Phase (Data Range)	#Samples (Range)	MAPE
Prefill ($sl \leq 32k$)	18 ($sl \leq 16k$)	9.1%
	30 ($sl \leq 32k$)	5.3%
Decode ($cl \leq 32k, bs \leq 256$)	308 ($cl \leq 16k, bs \leq 128$)	10.5%
	364 ($cl \leq 32k, bs \leq 256$)	7.7%

在检测到潜在的 SLO 违规后，调度程序会贪婪地搜索最佳配置并调用计算资源管理器（第 3.4 节）来重新分配 SM。

如图 11 所示，对于预填充调度程序，每个步骤启动固定数量的层，并同步 CPU（青色三角形）以做出后续调度决策。这使得能够对预填充进度进行细粒度控制，以快速适应系统波动。相反，解码调度程序将内核发布为单个 CUDA 图 [18]，以消除小内核的启动开销。

主要调度目标是在考虑解码 SLO 的同时优先考虑预填充，因为较短的预填充延迟会扩大解码批量大小并提高系统吞吐量 [6]。在并发操作期间，解码阶段提供满足 SLO 的最小 SM 计数。当最终的预填充层接近完成时，额外的 SM 被分配给解码阶段，以促进共同运行的预填充/解码和仅解码之间的平滑过渡。在重新配置期间，Bullet 通过在阶段之间部分共享 SM 而不是闲置未使用的资源来消除阶段间同步。尽管完美的非重叠分配具有挑战性，但逐层预填充调度将这种计算资源干扰限制在最小的、可预测的区域。

3.3.2 请求调度和资源供应。为了促进 SLO 感知调度，Bullet 监视定义为 $S = (ES, PS, RS)$ 的系统状态，其中 ES 表示执行状态（第 3.2.1 节）， PS 表示预填充进度， RS 表示每个请求延迟指标。预填充状态 PS 包括排队请求集合 Q 、进行中请求集合 P 和执行层计数 L_{exec} 。对于每个请求 i ，记录到达时间 a_i 和解码开始时间 d_i 。在时间 t ，预填充调度程序估计当前 ES 下 $P \cup Q$ 中所有请求的 GPU 执行时间，而解码调度程序预测下一步延迟并更新相应的 TPOT。然后，这些估计被写入全局元数据缓冲区，允许每个调度程序感知整体状态。

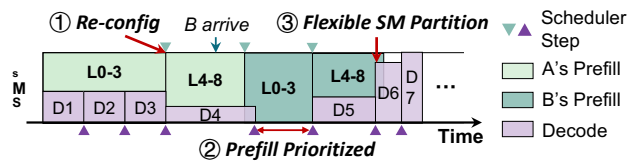


图 11. 异步调度程序启动分层预填充内核和逐步解码图，监控延迟并动态重新分配 SM。

算法 1 概述了预填充调度程序，解码调度程序也类似。该算法持续监控执行进度并更新延迟估计（第 2-4 行）。当重新排序不违反待处理请求的 TTFT SLO 时，等待队列中的请求将按预计延迟升序（第 5 行）重新排序。这会减少平均 TTFT 而不会出现饥饿现象。要开始新的预填充步骤（第 10-13 行），将对请求进行批处理，直到达到当前执行状态下的算术强度限制。在配置资源时，预填充阶段会优先考虑并配置更多 SM，除非 TPOT 受到损害（第 15 行）。在高请求负载的极端情况下，如果仍然满足 TPOT SLO，则解码阶段将暂时暂停（图 11-②）。当两个 SLO 不能同时得到满足时，表明系统超出了最大容量，则会强制执行平衡的 SM 比率，以限制任一阶段的过度延迟（第 14 行）。

3.4 计算资源管理器

虽然具有 CUDA Green Context [3] 的 MPS [39] 支持 SM 分区，但 LLM 服务 [19] 中仅 4 个静态策略的内存开销就超过 700MB，这使得 Bullet 的细粒度控制不切实际。为了实现灵活的低开销 SM 配置，我们采用了 SM 屏蔽技术库 [7, 8]。libsmctrl_set_stream_mask API 设置 CUDA 流 [18] 的元数据以限制所有内核

Algorithm 1: SLO-aware Scheduling for Prefill

Input: System state: S , TTFT/TPOT SLO: Γ_p/Γ_d

Output: Next requests & layers to run: $next_tasks$

```

1 Function Schedule( $S$ ):
2    $ttft \leftarrow \text{ESTIMATELATENCY}(S)$ 
3    $\text{WRITEGLOBALBUFFER}(ttft)$ 
4    $tpot \leftarrow \text{READGLOBALBUFFER}()$ 
5    $\text{SORTBYLEASTESTIMLATENCY}(Q)$ 
6   if  $P \neq \emptyset$  then
7      $satisfy \leftarrow \text{req}.ttft \leq \Gamma_p, \text{req} \in P$ 
8      $next\_tasks \leftarrow P$ 
9   else
10     $satisfy \leftarrow P90(ttft) \leq \Gamma_p$ 
11     $next\_tasks \leftarrow \emptyset$ 
12    while  $\text{ARITHINTEN}(next\_tasks, S.ES) < peak$  do
13       $next\_tasks.append(Q.pop())$ 
14  if not  $satisfy$  and  $P90(tpot) > \Gamma_d$  then
15     $\text{SETBALANCEDSM}(S, ttft, tpot)$ 
16  else if  $P90(tpot) \leq \Gamma_d$  then
17     $\text{REDUCEDECODESM}(S, ttft, tpot)$ 
18  else if  $P90(tpot) > \Gamma_d$  then
19     $\text{REDUCEPREFILLSM}(S, ttft, tpot)$ 
20  return  $next\_tasks, L_{exe} + L_{step}$ 

```

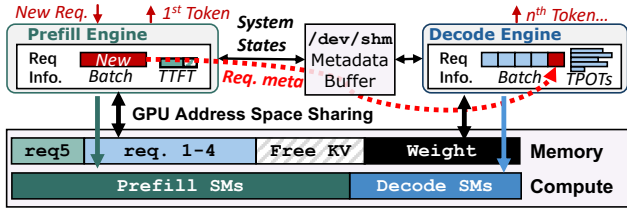


Figure 12. Concurrent execution engine with shared KV cache/weight GPU address spaces, exchanging system states and request metadata via OS-managed shared memory.

launched within it to execute on a specified SM subset. Section 4.3.3 validates that this approach adds only microsecond-level runtime overhead and zero extra memory. Similar interfaces exist on other platforms, exemplified by AMD’s `hipExtStreamCreateWithCUMask` [2].

Bullet creates a CUDA stream in each *concurrent execution engine* for prefill and decode. Whenever the scheduler issues a repartitioning command, the system immediately invokes the `libsmctrl` API to reconfigure the corresponding stream, thereby restricting all subsequent kernels to the newly provisioned SMs. This instantaneous configuration (Figure 11-①③), supports rapid adaptation to dynamic system states with flexible SM allocation.

3.5 Concurrent Execution Engine

3.5.1 Engine Architecture. Figure 12 illustrates the concurrent execution engines for prefill and decode, each residing in a separate process and driven by the corresponding scheduler. Both engines share a CPU buffer and unified GPU memory pool. The CPU buffer is implemented as OS-managed shared memory, stores global system states and employs compact control bits to indicate data availability, enabling low-latency status exchange. For GPU memory management, Bullet employs a dedicated initialization process that allocates model weights and KV cache [35] prior to engine launch. The resulting memory region is shared between engines via `cudaIpcGet/OpenMemHandle` API [18] with no adverse effects as documented. Equivalent facilities, such as AMD’s `hipIpcOpenMemHandle` [2], allow the same design to be deployed on other hardware. Since the address space and metadata are shared, Bullet is fully compatible with existing KV cache and prefix cache optimizations. An atomic lock serializes allocation and deallocation transactions that may be issued concurrently by both engines, ensuring correctness with minimal performance impact.

3.5.2 Execution Workflow. The prefill engine’s execution begins with receiving requests and follows the scheduling workflow in §3.3, retrieving decode-side states from the buffer, such as batch size and TPOTs. Once prefill completes, the request migrates to decode without KV-cache transfer, and only metadata is asynchronously sent to the

decode engine via ZeroMQ [55], enabling microsecond overhead. The decode engine receives newly prefilled requests and merges them into the current running batch. Output tokens are directly forwarded to the frontend server, eliminating any CPU involvement by the prefill engine. Bullet’s control plane works independently while proactively communicating through the buffer, unblocking both CPU and GPU execution. This decentralized architecture allows concurrent kernel submissions while eliminating the need for frequent synchronization compared with a centralized architecture. We implement Bullet on top of SGLang [58] v0.4.6 and PyTorch 2.6.0 with 4100 lines of Python code, and integrate a modified `libsmctrl` [7] library to optimize GPU resource allocation within the serving engine. The prefill and decode engines are implemented as SGLang’s workers, with MPS [39] enabled for spatial sharing.

4 Experimental Evaluation

4.1 Methodology

Evaluated Schemes. We compare Bullet against several state-of-the-art systems of chunked prefill-based optimization: vLLM [35] v0.8.5 (1024-chunk), SGLang [58] v0.4.6 (1024/2048-chunk), and Nanoflow [60] (1024-chunk).

Workloads Three representative datasets are used for evaluation, whose request arrivals follow a Poisson process, as widely adopted in prior works [6, 35, 60]. Figure 13 details the cumulative distribution function (CDF) of the datasets’ sequence length. The ShareGPT [48] contains real-world conversational data, Azure-Code [43] is production code completion trace released by Azure, and arXiv-Summary [14] is long-context summarization.

Models and Platforms. Experiments are conducted on three servers in Table 3 with CUDA 12.4. We evaluate dense model Llama3.1-8B/70B [24] and MoE model Qwen3-235B-A22B-FP8 [53]. All multi-GPU setups use tensor parallelism.

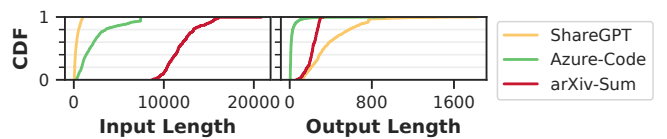


Figure 13. Workload input/output length CDF.

Table 3. Server configurations for experiments.

GPUs	Network Bw.	#SMs/GPU
8×A100-80GB	20 GB/s	108
8×H100	600 GB/s	132
8×H20	400 GB/s	78

Table 4. Workload latency requirements.

	ShareGPT	Azure-Code	arXiv-Sum
norm. TTFT	3.0 ms	1.5 ms	1.5 ms
TPOT	150 ms	200 ms	175 ms

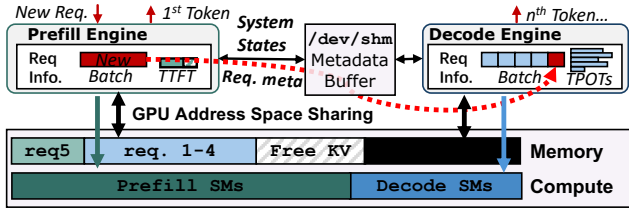


图 12. 具有共享 KV 缓存/权重 GPU 地址空间的并发执行引擎，通过操作系统管理的共享内存交换系统状态和请求元数据。

在其中启动以在指定的 SM 子集上执行。第 4.3.3 节验证了这种方法仅增加微秒级的运行时开销和零额外内存。其他平台上也存在类似的接口，例如 AMD 的 `hipExtStreamCreateWithCUMask` [2]。

Bullet 在每个并发执行引擎中创建一个 CUDA 流以进行预填充和解码。每当调度程序发出重新分区命令时，系统立即调用 `libsmctrl` API 来重新配置相应的流，从而将所有后续内核限制为新配置的 SM。这种瞬时配置（图 11-①③）支持通过灵活的 SM 分配快速适应动态系统状态。

3.5 并发执行引擎

3.5.1 引擎架构。图 12 说明了预填充和解码的并发执行引擎，每个引擎驻留在单独的进程中，并由相应的调度程序驱动。两个引擎共享 CPU 缓冲区和统一的 GPU 内存池。CPU 缓冲区被实现为操作系统管理的共享内存，存储全局系统状态并采用紧凑的控制位来指示数据可用性，从而实现低延迟状态交换。对于 GPU 内存管理，Bullet 采用专用初始化过程，在引擎启动之前分配模型权重和 KV 缓存 [35]。由此产生的内存区域通过 `cudaIpcGet/OpenMemHandle` API [18] 在引擎之间共享，并且没有记录的不利影响。等效的设施，例如 AMD 的 `hipIpcOpenMemHandle` [2]，允许将相同的设计部署在其他硬件上。由于地址空间和元数据是共享的，Bullet 与现有的 KV 缓存和前缀缓存优化完全兼容。原子锁序列化可能由两个引擎同时发出的分配和解除分配事务，从而确保正确性并同时性能影响降至最低。

3.5.2 执行工作流程。预填充引擎的执行从接收请求开始，并遵循第 3.3 节中的调度工作流程，从缓冲区检索解码端状态，例如批量大小和 TPOT。一旦预填充完成，请求就会迁移到解码，而无需 KV 缓存传输，并且只有元数据会异步发送到

通过 ZeroMQ [55] 进行解码引擎，实现微秒级开销。解码引擎接收新的预填充请求并将它们合并到当前正在运行的批次中。输出令牌直接转发到前端服务器，消除了预填充引擎的任何 CPU 参与。Bullet 的控制平面独立工作，同时通过缓冲区主动通信，从而解除 CPU 和 GPU 执行的阻塞。与集中式架构相比，这种分散式架构允许并发内核提交，同时消除了频繁同步的需要。我们在 SGLang [58] v0.4.6 和 PyTorch 2.6.0 之上使用 4100 行 Python 代码实现了 Bullet，并集成了修改后的 `libsmctrl` [7] 库以优化服务引擎内的 GPU 资源分配。预填充和解码引擎作为 SGLang 的工作线程实现，并启用 MPS [39] 来实现空间共享。

4 实验评价

4.1 方法论

评估方案。我们将 Bullet 与基于分块预填充优化的几种最先进的系统进行比较：vLLM [35] v0.8.5（1024-chunk）、SGLang [58] v0.4.6（1024/2048-chunk）和 Nanoflow [60]（1024-chunk）。

工作负载使用三个代表性数据集进行评估，其请求到达遵循泊松过程，如先前工作中广泛采用的那样 [6, 35, 60]。图 13 详细介绍了数据集序列长度的累积分布函数（CDF）。ShareGPT [48] 包含真实世界的对话数据，Azure-Code [43] 是 Azure 发布的生产代码完成跟踪，arXiv-Summary [14] 是长上下文摘要。

模型和平台。实验是在表 3 中的三台服务器上使用 CUDA 12.4 进行的。我们评估了密集模型 Llama3.1-8B/70B [24] 和 MoE 模型 Qwen3-235B-A22B-FP8 [53]。所有多 GPU 设置都使用张量并行性。

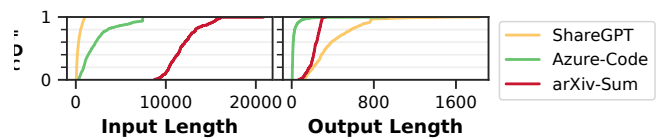


图 13. 工作负载输入/输出长度 CDF。

表 3. 实验的服务器配置。

GPUs	Network Bw.	#SMs/GPU
8×A100-80GB	20 GB/s	108
8×H100	600 GB/s	132
8×H20	400 GB/s	78

表 4. 工作负载延迟要求。

	ShareGPT	Azure-Code	arXiv-Sum
norm. TTFT	3.0 ms	1.5 ms	1.5 ms
TPOT	150 ms	200 ms	175 ms

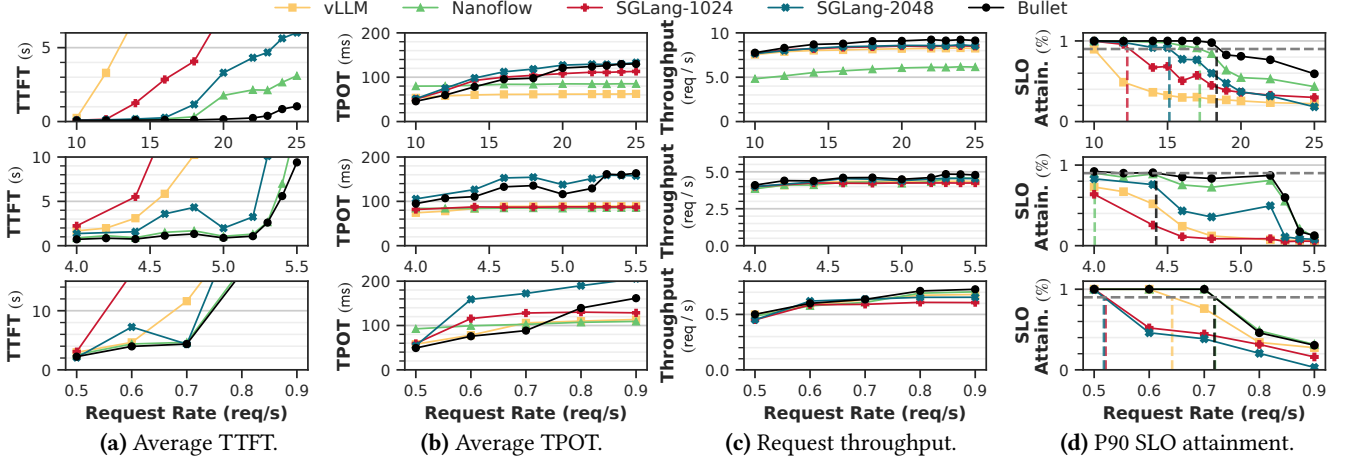


Figure 14. Performance comparison of ShareGPT (top row), Azure-Code (middle) and arXiv-Summary (bottom) datasets, evaluating average latency, throughput and SLO attainment rate of different LLM serving systems. Bullet achieves the highest throughput and SLO compliance in all workloads, attributed to remarkably decreased TTFT with modest TPOT increments.

Metrics. We report TTFT, TPOT and throughput. For SLO compliance measurement, we define as prefill and decode latency satisfying both the constraints listed in Table 4, aligned with previous works [51, 59, 60]. Due to the variation in sequence length, we use *normalized input/generation latency* established in previous works [51, 60] for SLO measurement. We collect hardware metrics using Nsight Systems [4].

4.2 End-to-End Performance

4.2.1 Single GPU Performance. Figure 14 evaluate Bullet against four baselines across three real-world workloads. Bullet demonstrates consistent throughput improvements, achieving 1.09× average and up to 1.20× higher throughput compared to SGLang-1024. Overall, Bullet maintains superior prefill latency enhancement (13.5×) with acceptable TPOT (0.94×) in average through dynamic SM allocation, translating to 1.86× end-to-end speedup compared with SGLang-1024. In contrast, all the chunked prefill-based systems exhibit unacceptable TTFT degradation even in a low request rate. For example, on the ShareGPT workload, with request rate of 20req/s, the prefill latency ranges from 1.8s (Nanoflow) to 14.9s (vLLM). The key reason is that chunk prefill limits the system’s capability to process the prefill phase (§2.3.2) and creates a cascading congestion for pending requests.

4.2.2 SLO Attainment. Figure 14d presents P90 SLO attainment rate. By eliminating these bottlenecks through intelligent SM allocation and adaptive scheduling, Bullet achieves mean TTFT of 0.16s and P90 tail latency of 0.31s, which is 54.9× and 78.5× better to SGLang-1024. The lower prefill latency remarkably translates to larger decode batches that boost overall throughput and SLO compliance (1.49×). Bullet effectively balances the throughput-latency tradeoff

that is skewed in chunked prefill systems. While SGLang-2048 demonstrates a 3.20× TTFT improvement and 1.18× higher throughput than SGLang-1024, the 2048 chunk suffers from 0.78× worse TPOT on average, confirming the imbalanced tradeoff inherent in chunked prefill. Bullet breaks this paradigm by simultaneously achieving both lower TTFT (4.2×) and better TPOT (1.20×) than SGLang-2048 through exploiting GPU utilization via concurrent prefill-decode execution. While Nanoflow achieves 2.37× longer TTFT and 0.86× shorter TPOT than Bullet via static kernel overlapping pipeline, it still suffers from the inherent limitations of chunked prefill that inflate TTFT tail latency. This results in 5.2% lower SLO compliance and 20.0% lower throughput compared to Bullet.

4.2.3 Multi-GPU Performance. Figure 15 depicts the generation latency of Bullet when Llama3.1-70B is deployed across eight A100 GPUs, with vLLM and SGLang using a chunk size of 2048. On the ShareGPT workload at 3.0 req/s, Bullet achieves a mean latency of 173 ms/token (223 ms/token for P99), whereas vLLM and SGLang attain 207 ms/token and 319 ms/token, respectively. For the Azure-Code dataset, where input lengths are significantly longer, Bullet’s advantages is more pronounced, sustaining 203 ms/token at

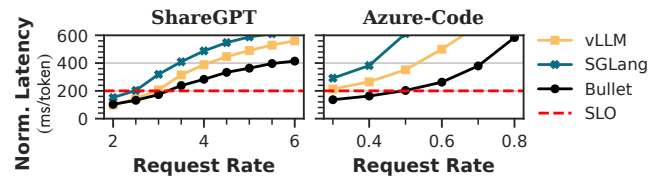


Figure 15. Mean normalized generation latency of Llama3.1-70B served on 8xA100. Bullet sustains higher request rates while remaining within the SLO constraint.

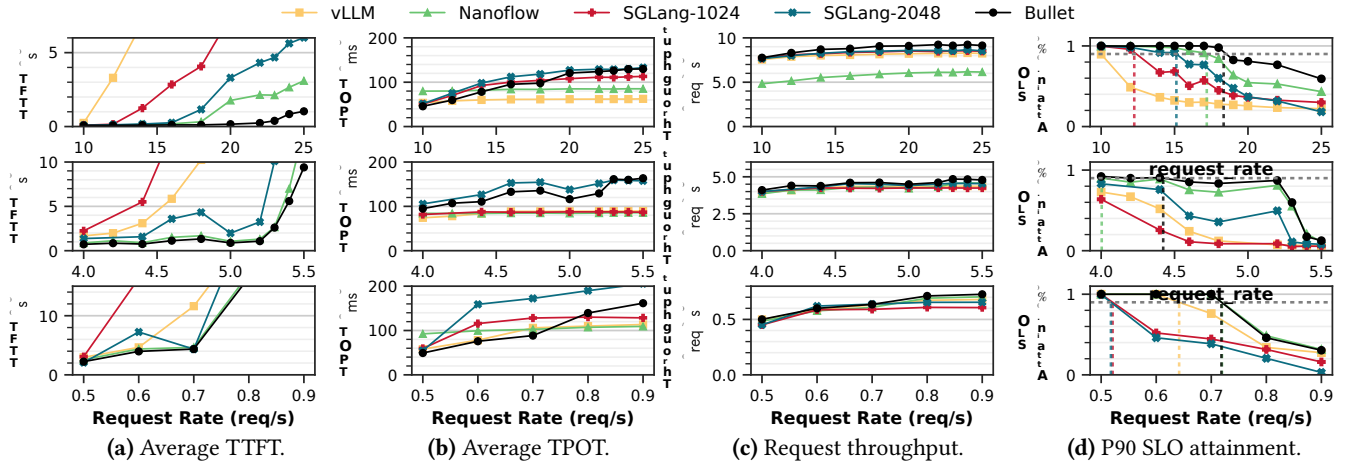


图 14. ShareGPT（顶行）、Azure-Code（中）和 arXiv-Summary（底）数据集的性能比较，评估不同 LLM 服务系统的平均延迟、吞吐量和 SLO 达到率。Bullet 在所有工作负载中实现了最高的吞吐量和 SLO 合规性，这归因于 TTFT 显著降低以及 TPOT 增量适度。

指标。我们报告 TTFT、TPOT 和吞吐量。对于 SLO 合规性测量，我们定义为满足表 4 中列出的约束的预填充和解码延迟，与之前的工作[51,59,60]保持一致。由于序列长度的变化，我们使用之前的工作 [51, 60] 中建立的归一化输入/生成延迟进行 SLO 测量。我们使用 Nsight Systems [4] 收集硬件指标。

4.2 端到端性能

4.2.1 单 GPU 性能。图 14 根据三个实际工作负载的四个基线评估 Bullet。Bullet 展示了持续的吞吐量改进，与 SGLang-1024 相比，平均吞吐量提高了 1.09 $\times$ ，吞吐量提高了 1.20 $\times$ 。总体而言，Bullet 通过动态 SM 分配保持了卓越的预填充延迟增强 (13.5 $\times$) 和可接受的平均 TPOT (0.94 $\times$)，与 SGLang-1024 相比，转化为 1.86 $\times$ 端到端加速。相比之下，所有基于分块预填充的系统即使在低请求率下也表现出不可接受的 TTFT 退化。例如，在 ShareGPT 工作负载上，请求率为 20req/s，预填充延迟范围为 1.8 秒 (Nanoflow) 到 14.9 秒 (vLLM)。关键原因是块预填充限制了系统处理预填充阶段（第 2.3.2 节）的能力，并有待处理的请求造成级联拥塞。

4.2.2 SLO 达到情况。图 14d 显示了 P90 SLO 达到率。通过智能 SM 分配和自适应调度消除这些瓶颈，Bullet 实现了 0.16 秒的平均 TTFT 和 0.31 秒的 P90 尾部延迟，比 SGLang-1024 好 54.9 $\times$ 和 78.5 $\times$ 。较低的预填充延迟显著转化为更大的解码批次，从而提高整体吞吐量和 SLO 合规性 (1.49 $\times$)。Bullet 有效地平衡了吞吐量与延迟的权衡

这在分块预填充系统中是倾斜的。虽然 SGLang-2048 与 SGLang-1024 相比，TTFT 提高了 3.20 $\times$ ，吞吐量提高了 1.18 $\times$ ，但 2048 块的 TPOT 平均差 0.78 $\times$ ，这证实了分块预填充固有的不平衡权衡。Bullet 打破了这一范式，通过并发预填充解码执行来利用 GPU 利用率，同时实现比 SGLang-2048 更低的 TTFT (4.2 $\times$) 和更好的 TPOT (1.20 $\times$)。虽然 Nanoflow 通过静态内核重叠管道实现了比 Bullet 长 2.37 $\times$ 的 TTFT 和短 0.86 $\times$ 的 TPOT，但它仍然受到分块预填充的固有限制，导致 TTFT 尾部延迟增大。与 Bullet 相比，这导致 SLO 合规性降低 5.2%，吞吐量降低 20.0%。

4.2.3 多 GPU 性能。图 15 描述了当 Llama3.1-70B 部署在 8 个 A100 GPU 上时 Bullet 的生成延迟，vLLM 和 SGLang 使用的块大小为 2048。在 3.0 req/s 的 ShareGPT 工作负载上，Bullet 的平均延迟为 173 毫秒/令牌 (P99 为 223 毫秒/令牌)，而 vLLM 和 SGLang 达到 207 分别为 ms/token 和 319 ms/token。对于输入长度明显更长的 Azure-Code 数据集，Bullet 的优势更加明显，在

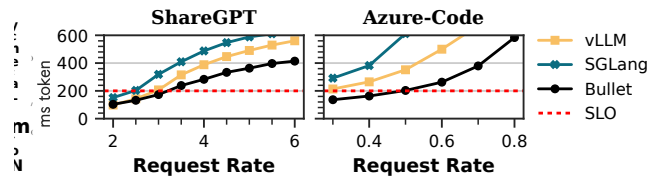


图 15. 在 8 $\times$ A100 上提供服务的 Llama3.1-70B 的平均标准化生成延迟。Bullet 维持较高的请求率，同时保持在 SLO 限制内。

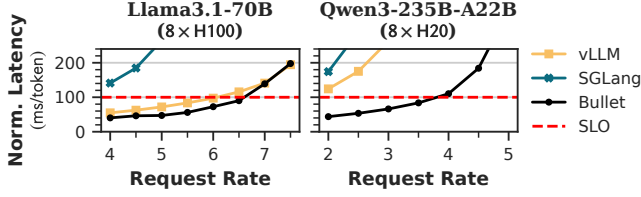


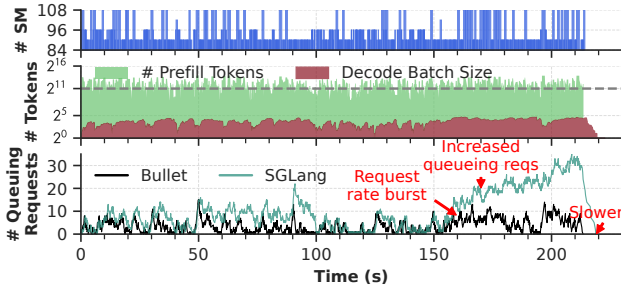
Figure 16. Mean normalized generation latency of various models across different hardware serving Azure-Code.

0.5 req/s. In contrast, vLLM and SGLang already exhibit 213 ms/token and 291 ms/token at a lower rate of 0.3 req/s.

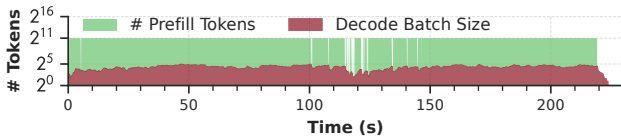
On H100 GPUs, the elevated memory bandwidth mitigates chunked-prefill overhead, yet Bullet continues outperforms SGLang and vLLM. Bullet’s concurrent handling of prefill and decode tasks leverages potential overlaps between computation, communication and memory operations with fine-grained SM control between phases. We also demonstrate Bullet’s generalization to other LLMs. For the highly sparse, FP8-quantized MoE Qwen3-235B-A22B, chunked prefill’s inherent lock-step batching impedes both prefill and decode speeds. However, Bullet orchestrates the two phases effectively. At 4.0 req/s, Bullet delivers 110 ms/token, with 1.4s TTFT and 45ms TPOT—respectively, which is 17.7 $\times$, 16.2 $\times$ and 4.2 $\times$ lower than vLLM.

4.3 Performance Breakdown

4.3.1 Dynamic SM Partition. We break down the effectiveness of dynamic resource provisioning in Figure 17 by timeline with the Azure-Code workload (5.0 req/s, Llama3.1-8B). The top row in Figure 17a demonstrates the number of SMs provisioned for the prefill phase, with each bar showing



(a) SM allocation for prefill changes dynamically according to system load (top). Concurrently processing tokens/batch size in Bullet (middle). Number of requests waiting for prefill (bottom). Bullet adaptively partition SMs to avoid request congestion on burst.



(b) SGLang’s hybrid batch status. Decode requests occupy chunk budget, degrade prefill efficiency, incurring severe queuing time.

Figure 17. System serving status of the Azure-Code workload in timeline view (request rate: 5.0 req/s).

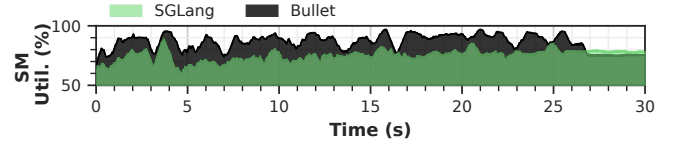


Figure 18. SM utilization of a timeslice in ShareGPT workload (20 req/s, Llama3.1-8B).

the SM count and duration. On request rate bursts (spikes in Figure 17-bottom), Bullet adaptively sets the number of prefill SMs to use full GPU, and may temporarily delay decode requests. This enables the rapid response for queuing requests, avoiding excessive waiting time. After processing the pending requests, Bullet quickly reconfigures the resources to a balance point for both phases. Instead, chunked prefill-based systems are not able to optimize such a scenario, since redundant KV cache access inflates the prefill speed. Moreover, decode batch shares the token budget with prefill tokens (Figure 17b), enforcing more iterations to complete a prefill, which further degrades performance. These factors jointly result in 4.17 $\times$ longer queuing delay for SGLang-2048. Bullet exempts from the token budget (Figure 17-middle) and orchestrates the two phases with fine-grained resource control to saturate GPU, significantly decreasing both TTFT and TPOT by 9.15 $\times$ and 1.33 $\times$, respectively.

4.3.2 GPU Utilization. We measure hardware utilization with Nsight Systems and present it in Figure 18. Between 0 s and 27 s, when the system concurrently handles prefill and decode requests, Bullet sustains an average of 86.2% active SM cycles, which is 11.2% higher than SGLang. Correspondingly, utilization of Tensor Cores rises by 11.8%, while memory-bandwidth utilization increases by 19.3%. These utilization gains directly reduce the mean end-to-end request latency from 25.8 s to 21.4 s.

4.3.3 Overheads. Table 5 quantifies the CPU overhead of Bullet’s key components. Sending and receiving meta-data only necessitates serialization and deserialization of Python objects, therefore achieving mean latency of 0.21ms. The performance prediction module incurs merely 10.2 μ s overhead by simply invoking the analytical model. For GPU resource management, SM masking enables instantaneous reconfiguration with negligible runtime overhead. These designs jointly ensure Bullet’s control plane adds minimal latency while enabling fine-grained resource optimization.

4.4 Sensitivity Studies

To study the effect of dynamic resource provisioning, we run the workloads under fixed SM configurations for prefill and allow decode to use all SMs. The results are reported in Figure 19. The SM-108 configuration (no partitioning) demonstrates severe imbalance in the Azure-Code workload. While achieving low TPOT, SM-108 suffers 1.20 $\times$ higher TTFT in average and 1.19 $\times$ worse P90 tail latency compared

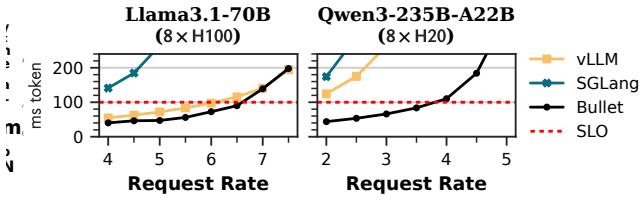


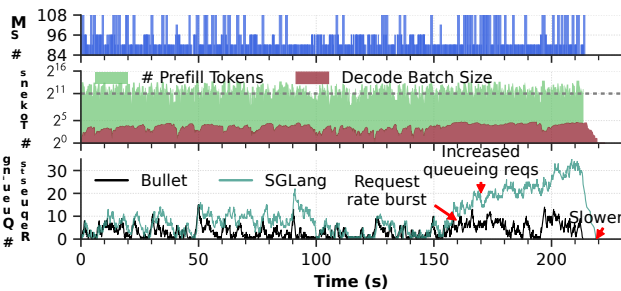
图 16. 跨不同硬件服务 Azure-Code 的各种模型的平均标准化生成延迟。

0.5 请求/秒。相比之下，vLLM 和 SGLang 已经以 0.3 req/s 的较低速率表现出 213 ms/token 和 291 ms/token。

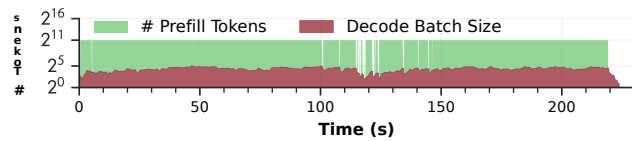
在 H100 GPU 上，提高的内存带宽减轻了分块预填充开销，但 Bullet 的性能仍然优于 SGLang 和 vLLM。Bullet 对预填充和解码任务的并发处理利用了计算、通信和内存操作之间的潜在重叠以及阶段之间的细粒度 SM 控制。我们还展示了 Bullet 对其他大语言模型。对于高度稀疏的 FP8 量化 MoE Qwen3-235B-A22B，分块预填充固有的锁步批处理会阻碍预填充和解码速度。然而，Bullet 有效地协调了这两个阶段。在 4.0 req/s 下，Bullet 交付 110 毫秒/令牌，TTFT 为 1.4 秒，TPOT 为 45 毫秒，比 vLLM 低 17.7×、16.2× 和 4.2×。

4.3 性能分解

4.3.1 动态 SM 分区。我们通过 Azure 代码工作负载（5.0 请求/秒，Llama3.1-8B）按时间线分解图 17 中动态资源配置的有效性。图 17a 中的顶行显示了为预填充阶段配置的 SM 数量，每个条形都显示



(a) 预填充的 SM 分配根据系统负载动态变化（上）。在 Bullet（中）中同时处理令牌/批量大小。等待预填充的请求数（底部）。子弹自适应分区 SM 以避免突发请求堵塞。



(b) SGLang 的混合批次状态。解码请求占用块预算，降低预填充效率，导致严重的排队时间。

图 17. 时间线视图中 Azure 代码工作负载的系统服务状态（请求率：5.0 请求/秒）。

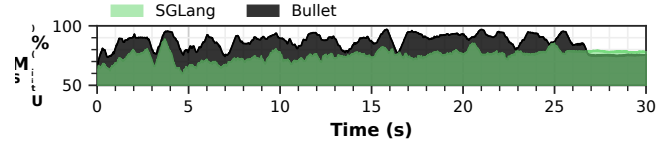


图 18. ShareGPT 工作负载中时间片的 SM 利用率（20 个请求/秒，Llama3.1-8B）。

SM 计数和持续时间。当请求率突发时（图 17 底部出现峰值），Bullet 会自适应地设置预填充 SM 的数量以使用完整的 GPU，并且可能会暂时延迟解码请求。这样可以快速响应排队请求，避免过多的等待时间。处理完待处理的请求后，Bullet 会快速将资源重新配置到两个阶段的平衡点。相反，基于分块预填充的系统无法优化这种情况，因为冗余的 KV 缓存访问会增加预填充速度。此外，解码批次与预填充令牌共享令牌预算（图 17b），强制执行更多迭代来完成预填充，这进一步降低了性能。这些因素共同导致 SGLang-2048 的排队延迟延长 4.17×。Bullet 免除了代币预算（图 17-中），并通过细粒度资源控制来协调两个阶段以使 GPU 饱和，从而使 TTFT 和 TPOT 分别显著降低 9.15× 和 1.33×。

4.3.2 GPU 利用率。我们使用 Nsight Systems 测量硬件利用率，并将其显示在图 18 中。在 0 秒到 27 秒之间，当系统同时处理预填充和解码请求时，Bullet 平均维持 86.2% 的活动 SM 周期，比 SGLang 高 11.2%。相应地，Tensor Core 的利用率增加了 11.8%，而内存带宽利用率则增加了 19.3%。这些利用率增益直接将平均端到端请求延迟从 25.8 秒减少到 21.4 秒。

4.3.3 管理费用。表 5 量化了 Bullet 关键组件的 CPU 开销。发送和接收元数据只需要 Python 对象的序列化和反序列化，因此平均延迟为 0.21ms。通过简单地调用分析模型，性能预测模块仅产生 10.2μs 开销。对于 GPU 资源管理，SM 屏蔽可实现即时重新配置，而运行时开销可以忽略不计。这些设计共同确保 Bullet 的控制平面增加最小的延迟，同时实现细粒度的资源优化。

4.4 敏感性研究

为了研究动态资源配置的效果，我们在固定 SM 配置下运行工作负载以进行预填充，并允许解码以使用所有 SM。结果如图 19 所示。SM-108 配置（无分区）表明 Azure 代码工作负载严重不平衡。在实现低 TPOT 的同时，SM-108 的 TTFT 平均高出 1.20×，P90 尾延迟差 1.19×

Table 5. Overheads in Bullet.

	Mean	Std.	P90	P99
Metadata Send/Recv (ms)	0.21	0.44	0.89	1.54
Performance Predict (μs)	10.2	5.1	24.5	25.8
Resource Re-config (μs)	4.1	0.79	4.2	5.9

to Bullet, ultimately reducing throughput and SLO attainment by 13%. Smaller static partitions, like 84 SMs, prove even more problematic, exacerbating latency imbalance with $1.78\times$ worse TTFT even than chunked prefill baselines and reducing throughput by 5.9%. Therefore, there is no optimal fixed SM allocation, as smaller partitions improve TPOT but degrade TTFT and introduce tail latency violations, and vice versa. These results confirm Bullet’s dynamic approach, which adaptively orchestrates resources to simultaneously maintain balanced latency targets and maximize throughput.

4.5 Ablation Studies

Figure 20 analyzes the contributions of different components in Bullet by isolating part of the design. Naive: concurrent execution without resource provisioning or scheduling. w/Partition: adds resource provision only. w/Scheduler: Incorporates only request reordering and delayed decode. The Naive design exhibits the expected latency imbalance (§4.4), high TPOT and low TTFT attributed to unpartitioned resource contention. The w/Partition variant improves TPOT for Azure-Code but suffers unacceptable TTFT degradation from its inability to reorder pending requests. Conversely, w/Scheduler maintains comparable latency while reducing Azure-Code TPOT through contention alleviation. Only the complete Bullet design, combining both partitioning and scheduling, achieves balanced latency across all workloads.

4.6 Discussions

While Bullet shows promising improvements, we identify several operational boundaries. First, on low-compute GPUs running dense models, the limited SM budget obliges the decode phase to claim a larger ratio of SMs to saturate memory bandwidth (Figure 8a), slightly tempering concurrent-execution benefits. However, this caveat is immaterial for

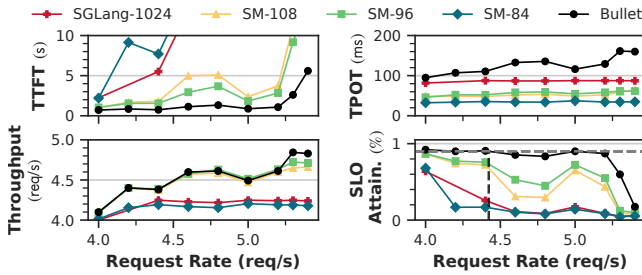


Figure 19. Sensitivity study on fixed SM counts for prefill. Static setups cause latency-throughput imbalance: more SMs cut TTFT but harm TPOT and goodput. Bullet’s dynamic SM tuning optimizes both.

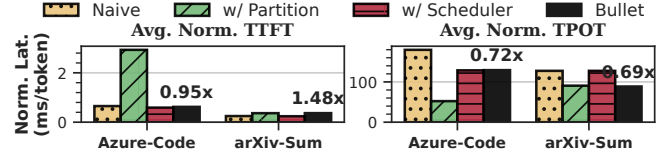


Figure 20. Ablation study of Bullet.

MoE models, whose lighter compute footprint preserves the full advantage (Figure 16). Second, Bullet’s SM isolation policy could be relaxed to partial sharing, yet such exploration is orthogonal to the paper’s core contribution of fine-grained spatial-temporal orchestration and can be retrofitted without architectural changes. Third, for specialized architectures like DeepSeek-MLA [21] where disaggregation proves fundamentally advantageous, Bullet’s co-located approach may not match dedicated solutions’ peak performance. Extending the latency model to cover new attention variants or LoRA adapters [32] is straightforward and left to future work. Bullet’s core innovations maintain broad applicability across standard LLM architectures and typical workload distributions while extensible to emerging models.

5 Related Works

LLM Serving Systems. System-level optimizations for LLM serving have been extensively explored, which can be categorized as prefill-decode co-location and disaggregation. Co-located techniques represented by chunked prefill [6, 60] improve batching efficiency but maintain lockstep prefill and decode phase execution that underutilizes hardware. Disaggregated systems [25, 43, 45, 59] eliminate inter-phase interference but incur costly KV cache migration and load imbalance among prefill-decode instances. Bullet uniquely enables intra-GPU prefill-decode execution, maximizing utilization without overheads while meeting SLOs. Recent intra-GPU sharing proposals for LLM inference remain fundamentally constrained. Nanoflow [60] exploits kernel overlap atop chunked prefill. However, the approach’s intrinsic limitations erode gains as sequence lengths increase. MuxServe [23] and Semi-PD [30] rely on static SM allocations, requiring costly engine relaunches for reconfiguration. Drift [19] explores lock-step execution between prefill blocks and decode in gangs using predefined SM partitions. These coarse designs lack the agility required by dynamic workloads. Bullet addresses these shortcomings through adaptive fine-grained resource management, enabling optimal SM utilization across diverse workloads.

Concurrent Kernel Execution. Co-executing kernels with complementary resource demands has been proven to improve GPU utilization and performance. Spatial-temporal multiplexing techniques [13, 22, 28] typically classify kernels by computational characteristics and rely on CUDA streams [18] or MPS [39] for kernel submission. However, these approaches treat the hardware scheduler as a blackbox, providing limited control [33, 40] over actual execution. While

表 5. 项目符号中的开销。

	Mean	Std.	P90	P99
Metadata Send/Recv (ms)	0.21	0.44	0.89	1.54
Performance Predict (μs)	10.2	5.1	24.5	25.8
Resource Re-config (μs)	4.1	0.79	4.2	5.9

到 Bullet，最终将吞吐量和 SLO 达到率降低了 13%。事实证明，较小的静态分区（例如 84 个 SM）的问题甚至更大，会加剧延迟不平衡，TTFT 甚至比分块预填充基线差 1.78x，并使吞吐量降低 5.9%。因此，不存在最佳的固定 SM 分配，因为较小的分区可以改善 TPOT，但会降低 TTFT 并引入尾部延迟违规，反之亦然。这些结果证实了 Bullet 的动态方法，该方法自适应地编排资源，以同时维持平衡的延迟目标并最大化吞吐量。

4.5 消融研究

图 20 通过隔离部分设计来分析 Bullet 中不同组件的贡献。Naive：无需资源配置或调度的并发执行。带分区：仅添加资源供应。带调度程序：仅包含请求重新排序和延迟解码。Naive 设计表现出预期的延迟不平衡（第 4.4 节）、由于未分区资源争用而造成的高 TPOT 和低 TTFT。w/Partition 变体改进了 Azure 代码的 TPOT，但由于无法对挂起的请求重新排序而遭受了不可接受的 TTFT 降级。相反，w/Scheduler 可保持相当的延迟，同时通过缓解争用来减少 Azure 代码 TPOT。只有结合分区和调度的完整 Bullet 设计才能在所有工作负载中实现平衡的延迟。

4.6 讨论

虽然 Bullet 显示了有希望的改进，但我们确定了几个操作边界。首先，在运行密集模型的低计算 GPU 上，有限的 SM 预算迫使解码阶段要求更大比例的 SM 来饱和和内存带宽（图 8a），从而稍微削弱了并发执行的优势。然而，这个警告对于

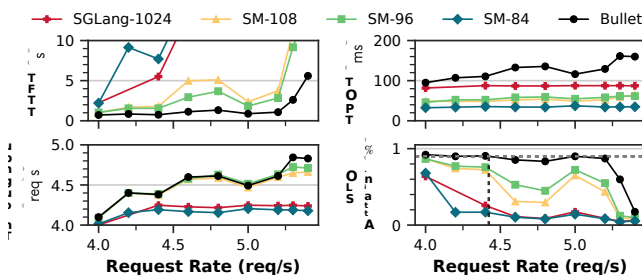


图 19. 预填充固定 SM 数量的敏感性研究。静态设置会导致延迟-吞吐量不平衡：更多 SM 会削减 TTFT，但会损害 TPOT 和良好吞吐量。Bullet 的动态 SM 调整对两者进行了优化。

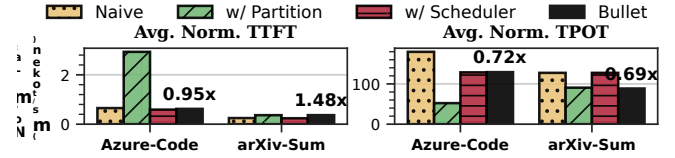


图 20. Bullet 的消融研究。

MoE 模型，其较轻的计算占用量保留了全部优势（图 16）。其次，Bullet 的 SM 隔离策略可以放宽到部分共享，但这种探索与论文的细粒度时空编排的核心贡献是正交的，并且可以在不进行架构更改的情况下进行改造。第三，对于像 DeepSeek-MLA [21] 这样的专业架构，分解被证明具有根本优势，Bullet 的共置方法可能无法与专用解决方案的峰值性能相匹配。扩展延迟模型以涵盖新的注意力变体或 LoRA 适配器 [32] 很简单，留待未来的工作。Bullet 的核心创新在标准大语言模型和典型工作负载分布中保持了广泛的适用性，同时可扩展到新兴模型。

5 相关作品

大语言模型系统。 LLM 服务的系统级优化已被广泛探索，可分为预填充解码共置和分解。以分块预填充 [6, 60] 为代表的共置技术提高了批处理效率，但维持了未充分利用硬件的锁步预填充和解码阶段执行。分解系统 [25,43,45,59] 消除了阶段间干扰，但会导致昂贵的 KV 缓存迁移和预填充解码实例之间的负载不平衡。Bullet 独特地支持 GPU 内预填充解码执行，在满足 SLO 的同时，在没有开销的情况下最大限度地提高利用率。最近针对 LLM 推理的 GPU 内共享提案仍然受到根本限制。Nanoflow [60] 利用分块预填充顶部的内核重叠。然而，随着序列长度的增加，该方法的内在局限性会削弱收益。MuxServe [23] 和 Semi-PD [30] 依赖于静态 SM 分配，需要昂贵的引擎重新启动才能进行重新配置。Drift [19] 探索了预填充块之间的锁步执行以及使用预定义的 SM 分区在组中进行解码。这些粗略的设计缺乏动态工作负载所需的敏捷性。Bullet 通过自适应细粒度资源管理解决了这些缺点，从而在不同的工作负载中实现最佳的 SM 利用率。

并发内核执行。事实证明，具有互补资源需求的共同执行内核可以提高 GPU 利用率和性能。时空复用技术 [13,22,28] 通常根据计算特征对内核进行分类，并依赖 CUDA 流 [18] 或 MPS [39] 进行内核提交。然而，这些方法将硬件调度程序视为黑匣子，对实际执行提供有限的控制 [33, 40]。尽管

several systems use precise kernel timing and resource provisioning [12, 46, 56] to enforce predictable scheduling, they focus on traditional DNN workloads and fail to address LLM-specific requirements, including prefill-decode phase coordination and KV cache management. Bullet distinguishes itself from previous works by fine-grained orchestration of LLM's phases and layer-level precise resource management.

Kernel Scheduling. Effective scheduling is essential for optimizing concurrent kernel execution. Latency model-based solutions [12, 46, 56] schedule kernels within latency constraints but rely on excessive offline profiling, which is unsuitable for the dynamic workloads in LLM serving. Although estimating standalone LLM inference latency is well-established [5, 51, 54], the complex interactions between concurrent prefill and decode phases remain unstudied. Existing solutions either rely on simplified models [30] or assume contention-free execution [19]. Bullet addresses this gap by developing a profile-augmented analytical model for performance estimation and refining kernel scheduling mechanisms for LLM serving.

6 Conclusion

This paper proposes Bullet to optimize LLM serving through precise GPU resource provisioning. By carefully orchestrating spatial-temporal execution between prefill and decode phases, Bullet effectively saturates wasted GPU resources. At runtime, Bullet employs real-time task progress estimation to optimize scheduling decisions and resource allocation, enabling precise and fine-grained control over layer-level resource quotas. Experimental evaluation confirms consistent latency compliance and substantial throughput gains.

References

- [1] 2022. LlamaIndex. https://github.com/jerryliu/llama_index.
- [2] 2025. AMD HIP Runtime API Documentation. <https://rocm.docs.amd.com/en/latest/>.
- [3] 2025. CUDA Driver API.
- [4] 2025. Nsight Systems Documentation. <https://docs.nvidia.com/nsight-systems/UserGuide/index.html>.
- [5] Amey Agrawal, Nitin Kedia, Jayashree Mohan, et al. 2024. VIDUR: A Large-Scale Simulation Framework for LLM Inference. In *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024*, Phillip B. Gibbons, Gennady Pekhimenko, and Christopher De Sa (Eds.). mlsys.org. https://proceedings.mlsys.org/paper_files/paper/2024/hash/b74a8de47d2b3c928360e0a011f48351-Abstract-Conference.html
- [6] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* (2023).
- [7] Joshua Bakita and James H. Anderson. 2023. Hardware Compute Partitioning on NVIDIA GPUs. In *29th IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS 2023, San Antonio, TX, USA, May 9-12, 2023*. IEEE, 54–66. doi:10.1109/RTAS58335.2023.00012
- [8] Joshua Bakita and James H. Anderson. 2024. Demystifying NVIDIA GPU Internals to Enable Reliable GPU Management. In *30th IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS 2024, Hong Kong, May 13-16, 2024*. IEEE, 294–305. doi:10.1109/RTAS61025.2024.00031
- [9] Joshua Bakita and James H Anderson. 2025. Hardware Compute Partitioning on NVIDIA GPUs for Composable Systems. In *37th Euromicro Conference on Real-Time Systems (ECRTS 2025)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 21–1.
- [10] Quan Chen, Hailong Yang, Jason Mars, and Lingjia Tang. 2016. Baymax: Qos awareness and increased utilization for non-preemptive accelerators in warehouse scale computers. *ACM SIGPLAN Notices* 51, 4 (2016), 681–696.
- [11] Ke Cheng, Zhi Wang, Wen Hu, Tiannuo Yang, Jianguo Li, and Sheng Zhang. [n. d.]. SCOOT: SLO-Oriented Performance Tuning for LLM Inference Engines. In *THE WEB CONFERENCE 2025*.
- [12] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. 2022. Serving heterogeneous machine learning models on {Multi-GPU} servers with {Spatio-Temporal} sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 199–216.
- [13] Marcus Chow, Ali Jahanshahi, and Daniel Wong. 2023. KRISP: Enabling Kernel-wise Right-sizing for Spatial Partitioned GPU Inference Servers. In *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2023, Montreal, QC, Canada, February 25 - March 1, 2023*. IEEE, 624–637. doi:10.1109/HPCA56546.2023.10071121
- [14] Arman Cohan, Franck Dernoncourt, Doo Soon Kim, et al. 2018. A Discourse-Aware Attention Model for Abstractive Summarization of Long Documents. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 2 (Short Papers)*, Marilyn A. Walker, Heng Ji, and Amanda Stent (Eds.). Association for Computational Linguistics, 615–621. doi:10.18653/V1/N18-2097
- [15] NVIDIA Corporation. [n. d.]. NVIDIA Deep Learning Performance. <https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html>.
- [16] Nvidia Corporation. 2021. Nvidia A100 tensor core GPU architecture. <https://resources.nvidia.com/en-us/genomics-ep/ampere-architecture-white-paper>.
- [17] NVIDIA Corporation. 2022. NVIDIA H100 Tensor Core GPU Architecture Overview. <https://resources.nvidia.com/en-us-tensor-core>.
- [18] NVIDIA Corporation. 2025. CUDA Runtime API Documentation. <https://docs.nvidia.com/cuda/cuda-runtime-api>.
- [19] Weihao Cui, Yukang Chen, Han Zhao, Ziyi Xu, Quan Chen, Xusheng Chen, Yangjie Zhou, Shixuan Sun, and Minyi Guo. 2025. Optimizing SLO-oriented LLM Serving with PD-Multiplexing. *arXiv preprint arXiv:2504.14489* (2025).
- [20] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arXiv:2205.14135 [cs.LG]* <https://arxiv.org/abs/2205.14135>
- [21] DeepSeek-AI, Aixin Liu, Bei Feng, et al. 2024. DeepSeek-V3 Technical Report. *CoRR abs/2412.19437* (2024). doi:10.48550/ARXIV.2412.19437 arXiv:2412.19437
- [22] Aditya Dhakal, Sameer G. Kulkarni, and K. K. Ramakrishnan. 2020. GSLICE: controlled spatial sharing of GPUs for a scalable inference platform. In *SoCC '20: ACM Symposium on Cloud Computing, Virtual Event, USA, October 19-21, 2020*, Rodrigo Fonseca, Christina Delimitrou, and Beng Chin Ooi (Eds.). ACM, 492–506. doi:10.1145/3419111.3421284
- [23] Jiangfei Duan, Runyu Lu, Haojie Duanmu, et al. 2024. MuxServe: Flexible Spatial-Temporal Multiplexing for Multiple LLM Serving. In *Forty-first International Conference on Machine Learning, ICLR 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net. <https://openreview.net/forum?id=R0SoZvqXyQ>
- [24] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, et al. 2024. The Llama 3 Herd of Models. *CoRR abs/2407.21783* (2024). doi:10.48550/ARXIV.2407.21783 arXiv:2407.21783

一些系统使用精确的内核定时和资源配置[12,46,56]来强制执行可预测的调度，它们专注于传统的DNN工作负载，无法满足LLM的特定要求，包括预填充解码阶段协调和KV缓存管理。Bullet与以往作品的区别在于LLM各个阶段的细粒度编排和层层精准的资源管理。

内核调度。有效的调度对于优化并发内核执行至关重要。基于延迟模型的解决方案[12,46,56]在延迟约束内核调度内核，但依赖于过多的离线分析，这不适合LLM服务中的动态工作负载。尽管估计独立LLM推理延迟已经很成熟[5,51,54]，但并发预填充和解码阶段之间的复杂交互仍未得到研究。现有的解决方案要么依赖于简化模型[30]，要么假设无争用执行[19]。Bullet通过开发用于性能评估的配置文件增强分析模型并完善LLM服务的内核调度机制来解决这一差距。

6 结论

本文提出Bullet通过精确的GPU资源配置来优化LLM服务。通过仔细编排预填充和解码阶段之间的时空执行，Bullet有效地饱和了浪费的GPU资源。在运行时，Bullet采用实时任务进度估计来优化调度决策和资源分配，从而实现对层级资源配额的精确和细粒度控制。实验评估证实了一致的延迟合规性和可观的吞吐量增益。

参考

[1] 2022. 骆驼指数。 https://github.com/jerryliu/llama_index. [2] 2025 年。AMD HIP 运行时 API 文档。 <https://rocm.docs.amd.com/en/latest/>。 [3] 2025. CUDA 驱动程序 API。 [4] 2025. Nsight 系统文档。 <https://docs.nvidia.com/nsight-systems/UserGuide/index.html>。 [5] Amey Agrawal, Nitin Kedia, Jayashree Mohan 等人。 2024. VIDUR: LLM 推理的大规模仿真框架。 第七届机器学习和系统年会记录, MLSys 2024, 美国加利福尼亚州圣克拉拉, 2024 年 5 月 13-16 日, Phillip B. Gibbons, Gennady Pekhimenko 和 Christopher De Sa (编辑)。 ml-sys.org。 https://proceedings.mlsys.org/paper_files/paper/2024/hash/b74a8de47d2b3c928360e0a011f48351-Abstract-Conference.html [6] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gula vani 和 Ramachandran Ramjee. 2023. Sarathi: 通过使用分块预填充进行解码来实现高效的 LLM 推理。 arXiv 预印本 arXiv:2308.16369 (2023)。 [7] 约书亚·巴基塔和詹姆斯·H·安德森。 2023 年。NVIDIA GPU 上的硬件计算分区。第 29 届 IEEE 实时和嵌入式技术与应用研讨会, RTAS 2023, 美国德克萨斯州圣安东尼奥, 2023 年 5 月 9-12 日。IEEE, 54-66. doi:10.1109/RTAS58335.2023.00012 [8] 约书亚·巴基塔和詹姆斯·H·安德森。 2024 年。揭秘 NVIDIA GPU 内部结构以实现可靠的 GPU 管理。第 30 届 IEEE 实时和嵌入式技术与应用研讨会, RTAS 2024, 香港, 2024 年 5 月 13-16 日。IEEE, 294-305. 号码: 10.1109/

RTAS61025.2024.00031 [9] 约书亚·巴基塔和詹姆斯·H·安德森。 2025 年。用于可组合系统的 NVIDIA GPU 上的硬件计算分区。第 37 届 Euromicro 实时系统会议 (ECRTS 2025)。达格斯图尔宫 - 莱布尼茨信息中心, 21-1。 [10] 陈泉, 杨海龙, 贾森·马尔斯, 唐凌嘉。 2016. Baymax: 仓库规模计算机中非抢占式加速器的 Qos 意识和利用率提高。ACM SIGPLAN 公告 51, 4 (2016), 681-696。 [11] 程柯, 王志, 胡文, 杨天诺, 李建国, 张胜。 [n. d.]. SCOOT: LLM 推理引擎的面向 SLO 的性能调优。 2025 年网络会议。 [12] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon 和 Jaehyuk Huh。 2022 年。通过(时空)共享在{多 GPU} 服务器上提供异构机器学习模型。 2022 年 USENIX 年度技术会议 (USENIX ATC 22)。 199-216。 [13] Marcus Chow, Ali Jahanshahi 和 Daniel Wong。 2023. KRISP: 为空间分区 GPU 推理服务器启用内核大小调整。 IEEE 国际高性能计算机架构研讨会, HPCA 2023, 加拿大魁北克省蒙特利尔, 2023 年 2 月 25 日至 3 月 1 日。IEEE, 624-637. doi:10.1109/HPCA56546.2023.10071121 [14] Arman Cohan, Franck Dernoncourt, Doo Soon Kim 等人。 2018. 用于长文档抽象总结的话语感知注意力模型。 计算语言学协会北美分会 2018 年会议记录: 人类语言技术, NAACL-HLT, 美国路易斯安那州新奥尔良, 2018 年 6 月 1-6 日, 第 2 卷 (短论文), Marilyn A. Walker, Heng Ji 和 Amanda Stent (编辑)。 计算语言学协会, 615-621. doi:10.18653/V1/N18-2097 [15] NVIDIA 公司。 [n. d.]. NVIDIA 深度学习性能。 <https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html>。 [16] 英伟达公司。 2021 年。Nvidia A100 张量核心 GPU 架构。 <https://resources.nvidia.com/en-us-genomics-ep/ampere-architecture-white-paper>。 [17] 英伟达公司。 2022 年。NVIDIA H100 Tensor Core GPU 架构概述。 <https://resources.nvidia.com/en-us-tensor-core>。 [18] 英伟达公司。 2025. CUDA 运行时 API 文档。 <https://docs.nvidia.com/cuda/cuda-runtime-api>。 [19] 崔伟豪, 陈玉康, 赵涵, 徐子怡, 陈全, 陈旭升, 周杨杰, 孙世轩, 郭敏仪。 2025. 使用 PD 复用优化面向 SLO 的 LLM 服务。 arXiv 预印本 arXiv:2504.14489 (2025)。 [20] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra 和 Christopher Ré。 2022. FlashAttention: 具有 IO 感知的快速、内存高效的精确注意力。 arXiv:2205.14135 [cs.LG] <https://arxiv.org/abs/2205.14135> [21] DeepSeek-AI, 刘爱新, 冯北, 等。 2024. DeepSeek-V3 技术报告。 CoRR 绝对/2412.19437 (2024)。 doi:10.48550/ARXIV.2412.19437 arXiv:2412.19437 [22] Aditya Dhakal, Sameer G. Kulkarni 和 K. K. Ramakrishnan。 2020. GSLICE: 用于可扩展推理平台的受控 GPU 空间共享。在 SoCC '20: ACM 云计算研讨会上, 虚拟活动, 美国, 2020 年 10 月 19 日至 21 日, Rodrigo Fonseca, Christina Delimitrou 和 Beng Chin Ooi (编辑)。 ACM, 492-506. doi:10.1145/3419111.3421284 [23] 段江飞, 路润玉, 端木浩杰, 等。 2024. MuxServe: 用于多个 LLM 服务的灵活时空复用。第四十一届机器学习国际会议, ICML 2024, 奥地利维也纳, 2024 年 7 月 21-27 日。OpenReview.net。 <https://openreview.net/forum?id=R0SoZvqXyQ> [24] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey 等人。 2024. Llama 3 模型群。 CoRR 绝对/2407.21783 (2024)。 doi:10.48550/ARXIV.2407.21783 arXiv:2407.21783

- [25] Bin Gao, Zhuomin He, Puru Sharma, et al. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with Cache-dAttention. In *Proceedings of the 2024 USENIX Annual Technical Conference, USENIX ATC 2024, Santa Clara, CA, USA, July 10-12, 2024*, Saurabh Bagchi and Yiying Zhang (Eds.). USENIX Association, 111–126. <https://www.usenix.org/conference/atc24/presentation/gao-bin-cost>
- [26] X Yu Geoffrey, Yubo Gao, Pavel Golikov, and Gennady Pekhimenko. 2021. Habitat: A {Runtime-Based} computational performance predictor for deep neural network training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 503–521.
- [27] Guin Gilman, Samuel S. Ogden, Tian Guo, and Robert J. Walls. 2020. Demystifying the Placement Policies of the NVIDIA GPU Thread Block Scheduler for Concurrent Kernels. *SIGMETRICS Perform. Evaluation Rev.* 48, 3 (2020), 81–88. doi:10.1145/3453953.3453972
- [28] Jianfeng Gu, Yichao Zhu, Puxuan Wang, et al. 2023. FaST-GShare: Enabling Efficient Spatio-Temporal GPU Sharing in Serverless Computing for Deep Learning Inference. In *Proceedings of the 52nd International Conference on Parallel Processing, ICPP 2023, Salt Lake City, UT, USA, August 7-10, 2023*. ACM, 635–644. doi:10.1145/3605573.3605638
- [29] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-scale Preemption for Concurrent GPU-accelerated DNN Inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, Marcos K. Aguilera and Hakim Weatherspoon (Eds.). USENIX Association, 539–558. <https://www.usenix.org/conference/osdi22/presentation/han>
- [30] Ke Hong, Lufang Chen, Zhong Wang, Xiuhong Li, Qiuli Mao, Jianping Ma, Chao Xiong, Guanyu Wu, Buhe Han, Guohao Dai, et al. 2025. semi-PD: Towards Efficient LLM Serving via Phase-Wise Disaggregated Computation and Unified Storage. *arXiv preprint arXiv:2504.19867* (2025).
- [31] Sirui Hong, Mingchen Zhuge, Jonathan Chen, et al. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. <https://openreview.net/forum?id=VtmBAGCN7o>
- [32] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [33] Paras Jain, Xiangxi Mo, Ajay Jain, et al. 2019. Dynamic Space-Time Scheduling for GPU Inference. *CoRR* abs/1901.00041 (2019). arXiv:1901.00041 <http://arxiv.org/abs/1901.00041>
- [34] Abhinav Jangda, Saeed Maleki, Maryam Mehri Dehnavi, Madan Musuvathi, and Olli Saarikivi. 2024. A Framework for Fine-Grained Synchronization of Dependent GPU Kernels. In *IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2024, Edinburgh, United Kingdom, March 2-6, 2024*, Tobias Grosser, Christophe Dubach, Michel Steuwer, Jingling Xue, Guilherme Ottoni, and ernando Magno Quintão Pereira (Eds.). IEEE, 93–105. doi:10.1109/CGO57630.2024.10444873
- [35] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace (Eds.). ACM, 611–626. doi:10.1145/3600006.3613165
- [36] Seonho Lee, Amar Phanishayee, and Divya Mahajan. 2025. Forecasting GPU performance for deep learning training and inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 493–508.
- [37] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, et al. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, 929–945. <https://www.usenix.org/conference/osdi24/presentation/lin-chaofan>
- [38] NVIDIA. 2023. TensorRT-LLM. <https://github.com/NVIDIA/TensorRT-LLM>.
- [39] NVIDIA. 2025. Multi-Process Service. <https://docs.nvidia.com/deploy/mps/index.html>.
- [40] Ignacio Sanudo Olmedo, Nicola Capodiceci, Jorge Luis Martinez, et al. 2020. Dissecting the CUDA scheduling hierarchy: a Performance and Predictability Perspective. In *IEEE Real-Time and Embedded Technology and Applications Symposium, RTAS 2020, Sydney, Australia, April 21-24, 2020*. IEEE, 213–225. doi:10.1109/RTAS48715.2020.000-5
- [41] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). doi:10.48550/ARXIV.2303.08774 arXiv:2303.08774
- [42] Muhammad Osama, Duane Merrill, Cris Cecka, et al. 2023. Stream-K: Work-Centric Parallel Decomposition for Dense Matrix-Matrix Multiplication on the GPU. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPoPP 2023, Montreal, QC, Canada, 25 February 2023 - 1 March 2023*, Maryam Mehri Dehnavi, Milind Kulkarni, and Sriram Krishnamoorthy (Eds.). ACM, 429–431. doi:10.1145/3572848.3577479
- [43] Pratyush Patel, Esha Choukse, Chaojie Zhang, et al. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*. IEEE, 118–132. doi:10.1109/ISCA59077.2024.00019
- [44] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, et al. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ASPLOS 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*, Lieven Eeckhout, Georgios Smaragdakis, Kaitai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach (Eds.). ACM, 1133–1150. doi:10.1145/3669940.3707256
- [45] Ruoyu Qin, Zheming Li, Weiran He, et al. 2025. Mooncake: Trading More Storage for Less Computation - A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies, FAST 2025, Santa Clara, CA, February 25-27, 2025*, Haryadi S. Gunawi and Vasily Tarasov (Eds.). USENIX Association, 155–170. <https://www.usenix.org/conference/fast25/presentation/qin>
- [46] Foteini Strati, Xianzhe Ma, and Ana Klimovic. 2024. Orion: Interference-aware, Fine-grained GPU Sharing for ML Applications. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*. ACM, 1075–1092. doi:10.1145/3627703.3629578
- [47] DeepSpeed Team. 2025. DeepSpeed-MII: Enabling Low-Latency, High-Throughput Inference. <https://github.com/deepspeedai/DeepSpeed-MII>.
- [48] ShareGPT Teams. 2023. Share your wildest ChatGPT conversations with one click. <https://sharegpt.com/>
- [49] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5998–6008. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [50] Zhenning Wang, Jun Yang, Rami G. Melhem, et al. 2016. Simultaneous Multikernel: Fine-Grained Sharing of GPUs. *IEEE Comput. Archit. Lett.* 15, 2 (2016), 113–116. doi:10.1109/LCA.2015.2477405
- [51] Bingyang Wu, Shengyu Liu, Yinmin Zhong, et al. 2024. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles, SOSP 2024, Austin, TX, USA*,

- [25] 高斌, 何卓敏, 普鲁·夏尔马, 等。2024. 具有 Cache-dAttention 的用于多轮对话的经济高效的大型语言模型。2024 年 USENIX 年度技术会议记录, USENIX ATC 2024, 美国加利福尼亚州圣克拉拉, 2024 年 7 月 10-12 日, Saurabh Bagchi 和 Yiyang Zhang (编辑)。USENIX 协会, 111-126。https://www.usenix.org/conference/atc24/presentation/gao-bin-cost [26] X Yu Geoffrey, Yubo Taka, Pavel Golikov 和 Gennady Pekhimenko。2021. Habitat: 用于深度神经网络训练的{基于运行时}的计算性能预测器。2021 年 USENIX 年度技术会议 (USENIX ATC 21)。503-521。[27] Guin Gilman, Samuel S. Ogden, Tian Jing 和 Robert J. Walls。2020. 揭秘并发内核的 NVIDIA GPU 线程块调度程序的放置策略。SIGMETRICS 执行。评估修订版 48, 3 (2020), 81-88。doi:10.1145/3453953.3453972 [28] 顾剑峰, 朱一超, 王璞轩, 等。2023. FaST-GShare: 在深度学习推理的无服务器计算中实现高效的时空 GPU 共享。第 52 届国际并行处理会议记录, ICPP 2023, 美国犹他州盐湖城, 2023 年 8 月 7-10 日。ACM, 635-644。doi:10.1145/3605573.3605638 [29] 韩明聪, 张汉泽, 陈榕, 陈海波。2022. 并发 GPU 加速 DNN 推理的微妙级抢占。第 16 届 USENIX 操作系统设计与实现研讨会, OSDI 2022, 美国加利福尼亚州卡尔斯巴德, 2022 年 7 月 11-13 日, Marcos K. Aguilera 和 Hakim Weatherspoon (编辑)。USENIX 协会, 539-558。https://www.usenix.org/conference/osdi22/presentation/han [30] 洪柯, 陈路芳, 王忠, 李秀红, 毛秋丽, 马建平, 熊超, 吴冠宇, 韩不和, 戴国豪, 等。2025. 半PD: 通过分阶段分解计算和统一存储实现高效的 LLM 服务。arXiv 预印本 arXiv:2504.19867 (2025)。[31] 洪思瑞, 诸葛名臣, 陈乔纳森, 等。2024. MetaGPT: 多代理协作框架的元编程。第十二届学习表征国际会议, ICLR 2024, 奥地利维也纳, 2024 年 5 月 7-11 日。OpenReview.net。https://openreview.net/forum?id=VtmBAGCN7o [32] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: 大型语言模型的低秩适应。ICLR 1, 2 (2022), 3。[33] Paras Jain, Xiangxi Mo, Ajay Jain 等人。2019. GPU 推理的动态时空调度。CoRR 绝对值/1901.00041 (2019)。arXiv:1901.00041 http://arxiv.org/abs/1901.00041 [34] Abhinav Jangda, Saeed Maleki, Maryam Mehri Dehnavi, Madan Musuvathi 和 Olli Saarikivi。2024. 依赖 GPU 内核细粒度同步的框架。在 IEEE/ACM 国际代码生成和优化研讨会上, CGO 2024, 英国爱丁堡, 2024 年 3 月 2-6 日, Tobias Grosser, Christophe Dubach, Michel Steuwer, Jingling Xu, Guilherme Ottoni 和 ernando Magno Quintão Pereira (编辑)。IEEE, 93-105。doi: 10.1109/CGO57630。2024.10444873 [35] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al。2023. 使用 PagedAttention 进行大型语言模型的高效内存管理。第 29 届操作系统原理研讨会论文集, SOSP 2023, 德国科布伦茨, 2023 年 10 月 23-26 日, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann 和 Jonathan Mace (编辑)。ACM, 611-626。doi:10.1145/3600006.3613165 [36] Seonho Lee, Amar Phanishaye 和 Divya Mahajan。2025 年。预测深度学习训练和推理的 GPU 性能。第 30 届 ACM 国际编程语言和操作系统架构支持会议论文集, 第 1 卷。493-508。[37] 林超凡, 韩振华, 张成瑞东, 等。2024. Parrot: 利用语义变量对基于 LLM 的应用程序进行高效服务。第 18 届 USENIX 操作系统设计与实现研讨会, OSDI 2024, 美国加利福尼亚州圣克拉拉, 2024 年 7 月 10-12 日, Ada 加夫里洛夫斯卡和道格拉斯·B 特里 (编)。USENIX 协会, 929-945。https://www.usenix.org/conference/osdi24/presentation/lin-chaofan [38] NVIDIA。2023. TensorRT-大语言模型。https://github.com/NVIDIA/TensorRT-LLM。[39] 英伟达。2025. 多进程服务。https://docs.nvidia.com/deploy/mps/index.html。[40] 伊格纳西奥·萨努多·奥尔梅多、尼古拉·卡波迪奇、豪尔赫·路易斯·马丁内斯等。2020. 剖析 CUDA 调度层次结构: 性能和可预测性视角。IEEE 实时和嵌入式技术与应用研讨会, RTAS 2020, 澳大利亚悉尼, 2020 年 4 月 21-24 日。IEEE, 213-225。doi:10.1109/RTAS48715.2020.000-5 [41] OpenAI。2023. GPT-4 技术报告。CoRR 绝对值/2303.08774 (2023)。doi:10.48550/ARXIV.2303.08774 arXiv:2303.08774 [42] Muhammad Osama, Duane Merrill, Cris Cecka 等人。2023. Stream-K: GPU 上密集矩阵-矩阵乘法的以工作为中心的并行分解。第 28 届 ACM SIGPLAN 并行编程原理与实践年度研讨会论文集, PPoPP 2023, 加拿大魁北克省蒙特利尔, 2023 年 2 月 25 日至 2023 年 3 月 1 日, Maryam Mehri Dehnavi, Milind Kulkarni 和 Sriram Krishnamoorthy (编辑)。ACM, 429-431。doi:10.1145/3572848.3577479 [43] Pratyush Patel, Esha Choukse, Chaojie Zhang, 等。2024. Splitwise: 使用相分裂的高效生成大语言模型 第 51 届 ACM/IEEE 计算机架构国际研讨会, ISCA 2024, 阿根廷布宜诺斯艾利斯, 2024 年 6 月 29 日至 7 月 3 日。IEEE, 118-132。doi:10.1109/ISCA59077.2024.00019 [44] Ramya Prabhu, Ajay Nayak, Jayashree Mohan 等人。2025. vAttention: 无需 PagedAttention 即可为 LLM 提供动态内存管理。第 30 届 ACM 国际编程语言和操作系统架构支持会议论文集, 第 1 卷, ASPLOS 2025, 荷兰鹿特丹, 2025 年 3 月 30 日至 2025 年 4 月 3 日, Lieven Eeckhout, Georgios Smaragdakis, Kaitai Liang, Adrian Sampson, Martha A. Kim 和 Christopher J. Rossbach (编辑)。ACM, 1133-1150。doi:10.1145/3669940.3707256 [45] 秦若宇, 李哲明, 何蔚然, 等。2025. Mooncake: 用更多的存储换取更少的计算 - 用于服务 LLM 聊天机器人的以 KVCache 为中心的架构。在第 23 届 USENIX 文件和存储技术会议 (FAST 2025) 上, Haryadi S. Gunawi 和 Vasily Tarasov (编辑) 于 2025 年 2 月 25 日至 27 日在加利福尼亚州圣克拉拉举行。USENIX 协会, 155-170。https://www.usenix.org/conference/fast25/presentation/qin [46] Foteini Strati, Xianzhe Ma 和 Ana Klimovic。2024. Orion: 用于 ML 应用程序的干扰感知、细粒度 GPU 共享。第十九届欧洲计算机系统会议记录, EuroSys 2024, 希腊雅典, 2024 年 4 月 22-25 日。ACM, 1075-1092。doi:10.1145/3627703.3629578 [47] DeepSpeed 团队。2025. DeepSpeed-MII: 实现低延迟、高吞吐量推理。https://github.com/deepspeedai/DeepSpeed-MII。[48] ShareGPT 团队。2023 年。一键分享您最疯狂的 ChatGPT 对话。https://sharegpt.com/ [49] Ashish Vaswani, Noam Shazeer, Niki Parmar 等。2017 年。您所需要的就是注意力。《神经信息处理系统进展 30: 2017 年神经信息处理系统年会》, 2017 年 12 月 4-9 日, 美国加利福尼亚州长滩, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan 和 Roman Garnett (编辑)。5998-6008。https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html [50] 王振宁, 杨俊, Rami G. Melhem, 等人。2016. 同时多内核: GPU 的细粒度共享。IEEE 计算。建筑师。莱特。15, 2 (2016), 113-116。doi:10.1109/LCA.2015.2477405 [51] 吴冰阳, 刘胜宇, 钟银民, 等。2024. LoongServe: 通过弹性序列并行高效服务长上下文大型语言模型。在 ACM SIGOPS 第 30 届操作系统原理研讨会论文集, SOSP 2024, 美国德克萨斯州奥斯汀,

- November 4-6, 2024, Emmett Witchel, Christopher J. Rossbach, Andrea C. Arpaci-Dusseau, and Kimberly Keeton (Eds.). ACM, 640–654. doi:10.1145/3694715.3695948
- [52] Ran Xu, Subrata Mitra, Jason Rahman, Peter Bai, Bowen Zhou, Greg Bronevetsky, and Saurabh Bagchi. 2018. Pythia: Improving datacenter utilization via precise contention prediction for multiple co-located workloads. In *Proceedings of the 19th International Middleware Conference*. 146–160.
 - [53] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
 - [54] Zhihang Yuan, Yuzhang Shang, Yang Zhou, et al. 2024. LLM Inference Unveiled: Survey and Roofline Model Insights. *CoRR* abs/2402.16363 (2024). doi:10.48550/ARXIV.2402.16363 arXiv:2402.16363
 - [55] ZeroMQ authors. [n. d.]. ZeroMQ: An open-source universal messaging library. <https://zeromq.org/>
 - [56] Shulai Zhang, Quan Chen, Weihao Cui, Han Zhao, Chunyu Xue, Zhen Zheng, Wei Lin, and Minyi Guo. 2025. Improving GPU Sharing Performance through Adaptive Bubbleless Spatial-Temporal Sharing. In *Proceedings of the Twentieth European Conference on Computer Systems*. 573–588.
 - [57] Yongkang Zhang, Haoxuan Yu, Chenxia Han, et al. 2025. SGDR: Software-Defined Dynamic Resource Control for Concurrent DNN Inference on NVIDIA GPUs. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPoPP 2025, Las Vegas, NV, USA, March 1-5, 2025*. ACM, 267–281. doi:10.1145/3710848.3710863
 - [58] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, et al. 2023. Efficiently Programming Large Language Models using SGLang. *CoRR* abs/2312.07104 (2023). doi:10.48550/ARXIV.2312.07104 arXiv:2312.07104
 - [59] Yinmin Zhong, Shengyu Liu, Junda Chen, et al. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, Ada Gavrilovska and Douglas B. Terry (Eds.). USENIX Association, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
 - [60] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, et al. 2025. {NanoFlow}: Towards Optimal Large Language Model Serving Throughput. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 749–765.

2024 年 11 月 4 日至 6 日, Emmett Witchel、Christopher J. Rossbach、Andrea C. Arpaci-Dusseau 和 Kimberly Keeton (编辑)。ACM, 640–654。doi:10.1145/3694715.3695948 [52] Ran Xu、Subrata Mitra、Jason Rahman、Peter Bai、Bowen Zhou、Greg Bronevetsky 和 Saurabh Bagchi。2018.Pythia: 通过对多个共置工作负载进行精确的争用预测来提高数据中心利用率。第 19 届国际中间件会议论文集。146–160。[53] 安阳, 李安峰, 杨宝松, 张北辰, 惠斌源, 郑波, 于博文, 高昌, 黄承恩, 吕晨旭, 等。2025.Qwen3 技术报告。arXiv 预印本 arXiv:2505.09388 (2025)。[54] 袁志航, 尚玉章, 周扬, 等。2024. 法学硕士推理揭晓: 调查和屋顶线模型见解。CoRR 绝对/2402.16363 (2024)。doi:10.48550/ARXIV.2402.16363 arXiv:2402.16363 [55] ZeroMQ 作者。[n. d.]. ZeroMQ: 一个开源通用消息传递库。https://zeromq.org/ [56] 张舒来, 陈泉, 崔伟豪, 赵涵, 薛春雨, 郑振, 林伟, 郭敏仪。2025. 通过自适应无泡时空共享提高 GPU 共享性能。在第二十届欧洲计算机系统会议的会议记录中。573–588。

[57] 张永康, 于浩轩, 韩晨霞, 等。2025. SGDRC: NVIDIA GPU 上并发 DNN 推理的软件定义动态资源控制。第 30 届 ACM SIGPLAN 并行编程原理与实践年度研讨会论文集, PPoPP 2025, 美国内华达州拉斯维加斯, 2025 年 3 月 1-5 日。ACM, 267–281。土井: 10. 1145/3710848.3710863 [58] 郑联民, 尹良胜, 谢志强, 等。2023. 使用 SG Lang 高效编程大型语言模型。CoRR 绝对/2312.07104 (2023)。doi:10.48550/ARXIV.2312.07104 arXiv:2312.07104 [59] 钟银民, 刘胜宇, 陈俊达, 等。2024. DistServe: 分解预填充和解码以实现 Goodput 优化的大型语言模型服务。第 18 届 USENIX 操作系统设计与实现研讨会, OSDI 2024, 美国加利福尼亚州圣克拉拉, 2024 年 7 月 10-12 日, Ada Gavrilovska 和 Douglas B. Terry (主编)。USENIX 协会, 193–210。https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin [60] 朱侃, 高宇飞, 赵一龙, 赵良宇, 左格飞, 谷一乐, 谢德东, 叶子浩, 镰堀圭介, 林千羽, 等。2025. {NanoFlow}: 迈向最佳大型语言模型服务吞吐量。第 19 届 USENIX 操作系统设计与实现研讨会 (OSDI 25)。749–765。